SQL - Funkcje okna (Window functions)
Lab 1

Imiona i nazwiska: Karolina Węgrzyn, Patrycja Markiewicz

Celem ćwiczenia jest przygotowanie środowiska pracy, wstępne zapoznanie się z działaniem funkcji okna (window functions) w SQL, analiza wydajności zapytań i porównanie z rozwiązaniami przy wykorzystaniu "tradycyjnych" konstrukcji SQL

Swoje odpowiedzi wpisuj w miejsca oznaczone jako:

> Wyniki:

Ważne/wymagane są komentarze.

Zamieść kod rozwiązania oraz zrzuty ekranu pokazujące wyniki, (dołącz kod rozwiązania w formie tekstowej/źródłowej)

Zwróć uwagę na formatowanie kodu

Oprogramowanie - co jest potrzebne?

Do wykonania ćwiczenia potrzebne jest następujące oprogramowanie:

- MS SQL Server - wersja 2019, 2022, 2025
- PostgreSQL - wersja 15/16/17/18
- SQLite
- Narzędzia do komunikacji z bazą danych
  - SSMS - Microsoft SQL Managment Studio
  - DtataGrip lub DBeaver
- Przykładowa baza Northwind
  - W wersji dla każdego z wymienionych serwerów

Oprogramowanie dostępne jest na przygotowanej maszynie wirtualnej

Dokumentacja/Literatura

- Kathi Kellenberger, Clayton Groom, Ed Pollack, Expert T-SQL Window Functions in SQL Server 2019, Apres 2019

- Itzik Ben-Gan, T-SQL Window Functions: For Data Analysis and Beyond, Microsoft 2020

- Kilka linków do materiałów które mogą być pomocne - https://learn.microsoft.com/en-us/sql/t-sql/queries/select-over-clause-transact-sql?view=sql-server-ver16

  - https://www.sqlservertutorial.net/sql-server-window-functions/

    - https://www.sqlshack.com/use-window-functions-sql-server/
    - https://www.postgresql.org/docs/current/tutorial-window.html
    - https://www.postgresqltutorial.com/postgresql-window-function/
    - https://www.sqlite.org/windowfunctions.html
    - https://www.sqlitetutorial.net/sqlite-window-functions/

- W razie potrzeby - opis Ikonek używanych w graficznej prezentacji planu zapytania w SSMS jest tutaj:
    - https://docs.microsoft.com/en-us/sql/relational-databases/showplan-logical-and-physical-operators-reference

Przygotowanie

Uruchom SSMS - Skonfiguruj połączenie z bazą Northwind na lokalnym serwerze MS SQL

Uruchom DataGrip (lub Dbeaver)

- Skonfiguruj połączenia z bazą Northwind3
    - na lokalnym serwerze MS SQL
    - na lokalnym serwerze PostgreSQL
    - z lokalną bazą SQLite

Zadanie 1 - obserwacja

Wykonaj i porównaj wyniki następujących poleceń.

```
select avg(unitprice) avgprice
from products p;

select avg(unitprice) over () as avgprice
from products p;

select categoryid, avg(unitprice) avgprice
from products p
group by categoryid

select avg(unitprice) over (partition by categoryid) as avgprice
from products p;
```

Jaka jest są podobieństwa, jakie różnice pomiędzy grupowaniem danych a działaniem funkcji okna?

> Wyniki:

```
select avg(unitprice) avgprice
from products p;
```

	avgprice
1	28.83389609200614

Jedna średnia z wartości unitprice dla całej tabeli products. Grupuje dane.

```
select avg(unitprice) over () as avgprice
from products p;
```

	avgprice
77	28.83389609200614

Ta sama średnia z całej tabeli products, ale przez funkcję okienkową jest tyle wierszy ile w products. Nie grupuje danych jak avg().

```
select categoryid, avg(unitprice) avgprice
from products p
group by categoryid;
```

	categoryid	avgprice
1	8	20.682499885559082
2	7	32.369999694824216
3	1	37.979166666666664
4	5	20.25
5	4	28.729999923706053
6	2	22.854166825612385
7	6	54.00666666030884
8	3	25.1600000674908

Średnia jest liczona dla każdej kategorii osobno, ale przez grupowanie jest jeden wiersz na jedną kategorię.

```
select avg(unitprice) over (partition by categoryid) as avgprice
from products p;
```

	avgprice
60	54.00666666030884
61	32.369999694824216
62	32.369999694824216
63	32.369999694824216
64	32.369999694824216
65	32.369999694824216
66	20.682499885559082
67	20.682499885559082
68	20.682499885559082
69	20.682499885559082
70	20.682499885559082
71	20.682499885559082
72	20.682499885559082
73	20.682499885559082
74	20.682499885559082
75	20.682499885559082
76	20.682499885559082
77	20.682499885559082

Średnia liczona dla każdej kategorii osobno, ale nie ma grupowania danych, więc jest tyle wierszy, ile w początkowej tabeli products.

Zadanie 2 - obserwacja

Wykonaj i porównaj wyniki następujących poleceń.

```
--1)

select p.productid, p.ProductName, p.unitprice,
       (select avg(unitprice) from products) as avgprice
from products p
where productid < 10

--2)
select p.productid, p.ProductName, p.unitprice,
       avg(unitprice) over () as avgprice
from products p
where productid < 10
```

Jaka jest różnica? Czego dotyczy warunek w każdym z przypadków? Napisz polecenie równoważne:

- 1. z wykorzystaniem funkcji okna
- 2. z wykorzystaniem podzapytania

> Wyniki:

```
select p.productid, p.ProductName, p.unitprice,
       (select avg(unitprice) from products) as avgprice
from products p
where productid < 10
```

	productid	productname	unitprice	avgprice
1	1	Chai	18	28.83389609200614
2	2	Chang	19	28.83389609200614
3	3	Aniseed Syrup	10	28.83389609200614
4	4	Chef Anton's Cajun Seasoning	22	28.83389609200614
5	5	Chef Anton's Gumbo Mix	21.35	28.83389609200614
6	6	Grandma's Boysenberry Spread	25	28.83389609200614
7	7	Uncle Bob's Organic Dried Pears	30	28.83389609200614
8	8	Northwoods Cranberry Sauce	40	28.83389609200614
9	9	Mishi Kobe Niku	97	28.83389609200614

Warunek `WHERE` dotyczy tylko zapytania zewnętrznego, czyli decyduje o tym, które wiersze ostatecznie zostaną zwrócone. Podzapytanie wewnętrzne nie ma żadnego własnego warunku `WHERE`, dlatego średnia jest obliczana dla wszystkich produktów w tabeli.

Polecenie równoważne z wykorzystaniem funkcji okna:

```sql
with avg_all as (
    select productid, productname, unitprice,
           avg(unitprice) over () as avgprice
    from products
)
select * from avg_all
where productid < 10
```

	productid	productname	unitprice	avgprice
1	1	Chai	18	28.83389609200614
2	2	Chang	19	28.83389609200614
3	3	Aniseed Syrup	10	28.83389609200614
4	4	Chef Anton's Cajun Seasoning	22	28.83389609200614
5	5	Chef Anton's Gumbo Mix	21.35	28.83389609200614
6	6	Grandma's Boysenberry Spread	25	28.83389609200614
7	7	Uncle Bob's Organic Dried Pears	30	28.83389609200614
8	8	Northwoods Cranberry Sauce	40	28.83389609200614
9	9	Mishi Kobe Niku	97	28.83389609200614

```sql
select p.productid, p.ProductName, p.unitprice,
       avg(unitprice) over () as avgprice
from products p
where productid < 10
```

	productid	productname	unitprice	avgprice
1	1	Chai	18	31.372222264607746
2	2	Chang	19	31.372222264607746
3	3	Aniseed Syrup	10	31.372222264607746
4	4	Chef Anton's Cajun Seasoning	22	31.372222264607746
5	5	Chef Anton's Gumbo Mix	21.35	31.372222264607746
6	6	Grandma's Boysenberry Spread	25	31.372222264607746
7	7	Uncle Bob's Organic Dried Pears	30	31.372222264607746
8	8	Northwoods Cranberry Sauce	40	31.372222264607746
9	9	Mishi Kobe Niku	97	31.372222264607746

Funkcje okna są obliczane pod sam koniec przetwarzania zapytania, w szczególności po klauzuli `WHERE`, która filtruje wiersze przed uruchomieniem tej funkcji. Dlatego wartość `avgprice` jest tu średnią ceną wyłącznie dla produktów o id mniejszym niż 10.

Polecenie równoważne z wykorzystaniem podzapytania:

```sql
select p.productid, p.ProductName, p.unitprice,
       (select avg(unitprice) from products where productid < 10) as avgprice
from products p
where productid < 10
```

	productid	productname	unitprice	avgprice
1	1	Chai	18	31.372222264607746
2	2	Chang	19	31.372222264607746
3	3	Aniseed Syrup	10	31.372222264607746
4	4	Chef Anton's Cajun Seasoning	22	31.372222264607746
5	5	Chef Anton's Gumbo Mix	21.35	31.372222264607746
6	6	Grandma's Boysenberry Spread	25	31.372222264607746
7	7	Uncle Bob's Organic Dried Pears	30	31.372222264607746
8	8	Northwoods Cranberry Sauce	40	31.372222264607746
9	9	Mishi Kobe Niku	97	31.372222264607746

Zadanie 3

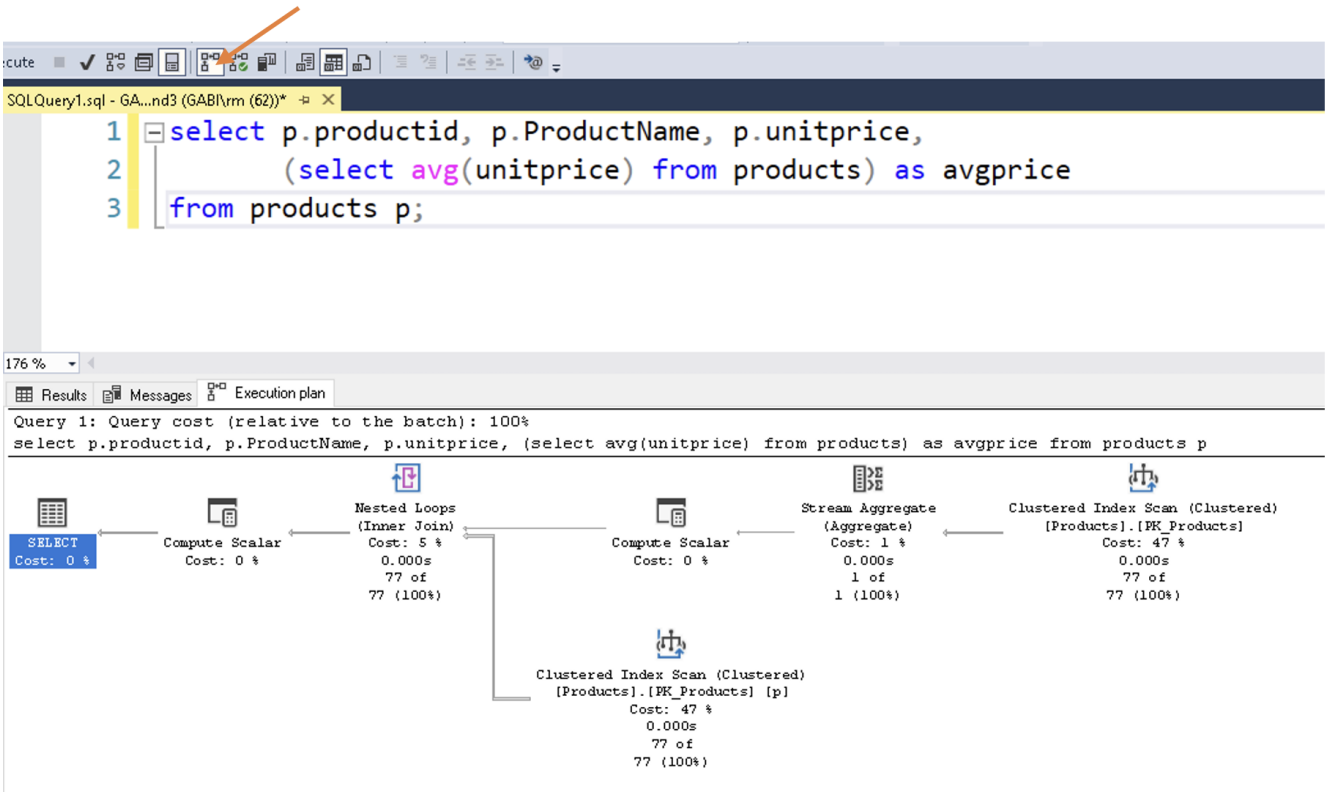
Baza: Northwind, tabela: products

Napisz polecenie, które zwraca: id produktu, nazwę produktu, cenę produktu, średnią cenę wszystkich produktów.

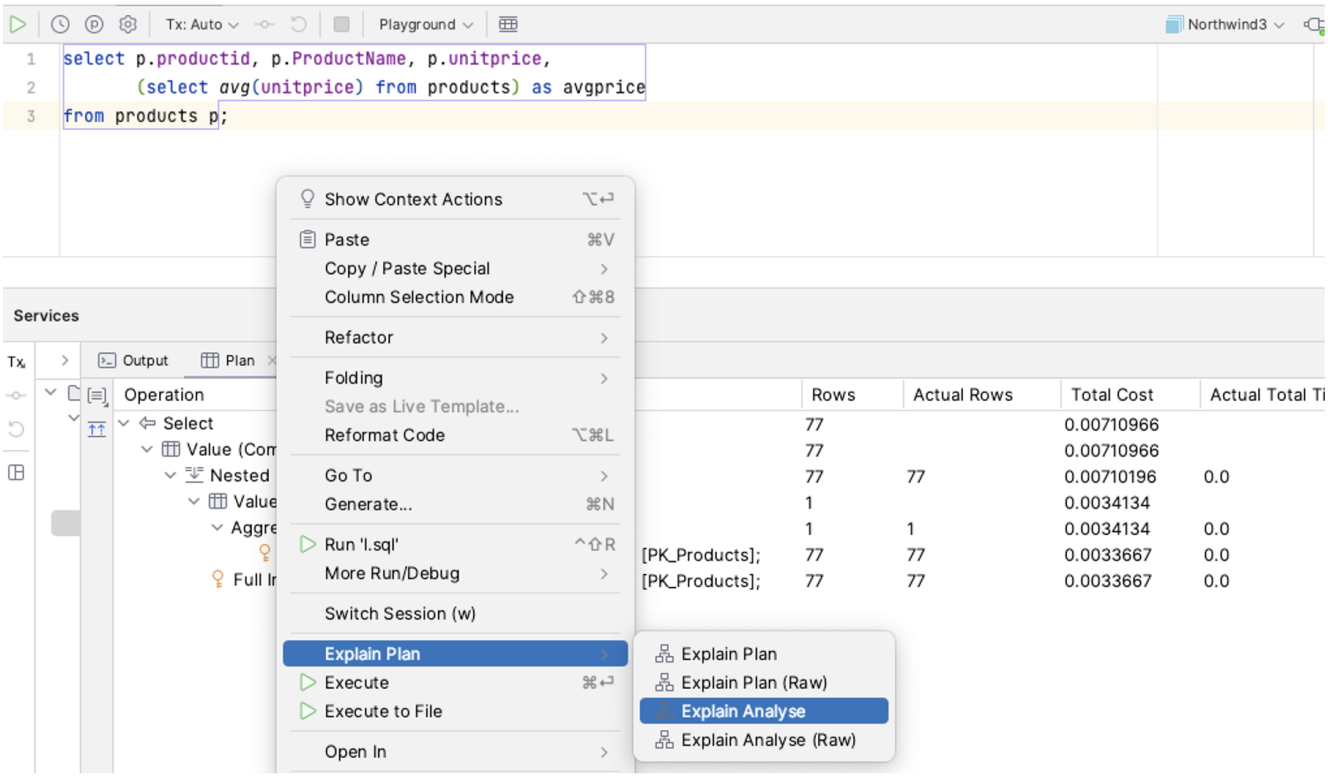
Napisz polecenie z wykorzystaniem z wykorzystaniem podzapytania, join'a oraz funkcji okna. Porównaj czasy oraz plany wykonania zapytań.

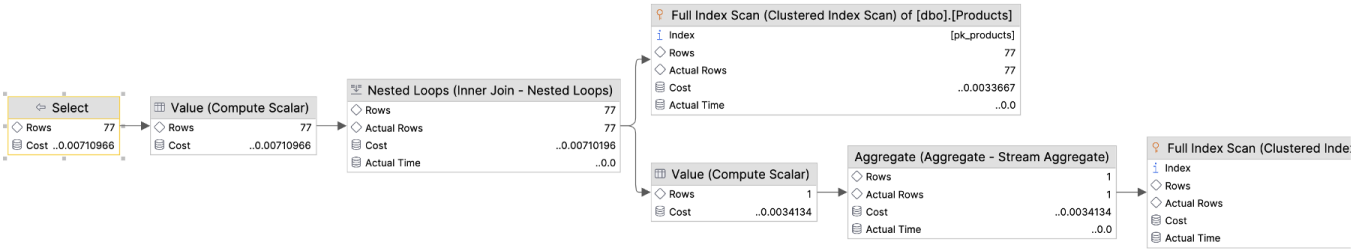
Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSql, SQLite)

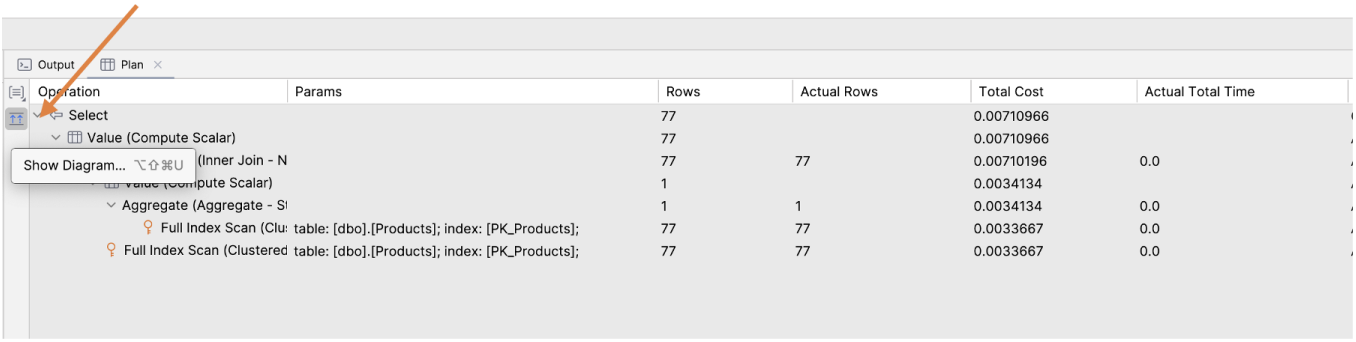
W SSMS włącz dwie opcje: Include Actual Execution Plan oraz Include Live Query Statistics



W DataGrip użyj opcji Explain Plan/Explain Analyze







> Wyniki:

```
--- 1
SELECT
    ProductID,
    ProductName,
    UnitPrice,
```

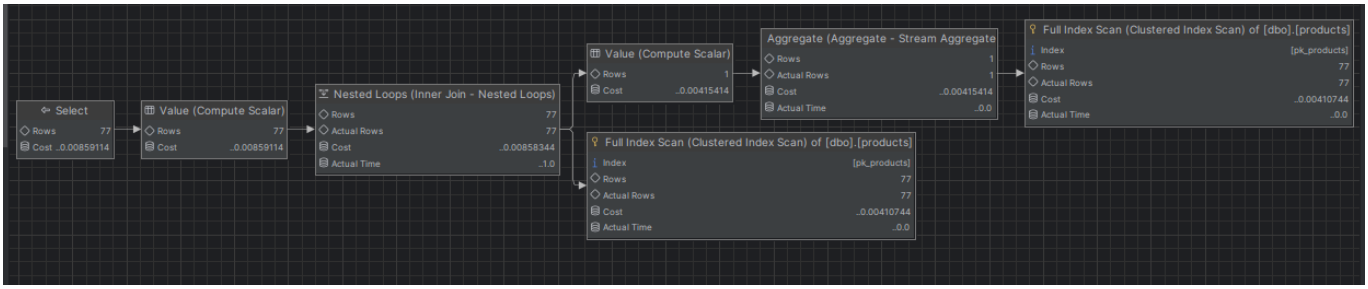
```
    (SELECT AVG(UnitPrice) FROM Products) AS AvgPrice
FROM Products;
```

```
--- 2
SELECT
    p.ProductID,
    p.ProductName,
    p.UnitPrice,
    avg_table.AvgPrice
FROM Products p
        CROSS JOIN (
    SELECT AVG(UnitPrice) AS AvgPrice
    FROM Products
) avg_table;
```

```
--- 3
SELECT
    ProductID,
    ProductName,
    UnitPrice,
    AVG(UnitPrice) OVER () AS AvgPrice
FROM Products;
```

MS SQL Server

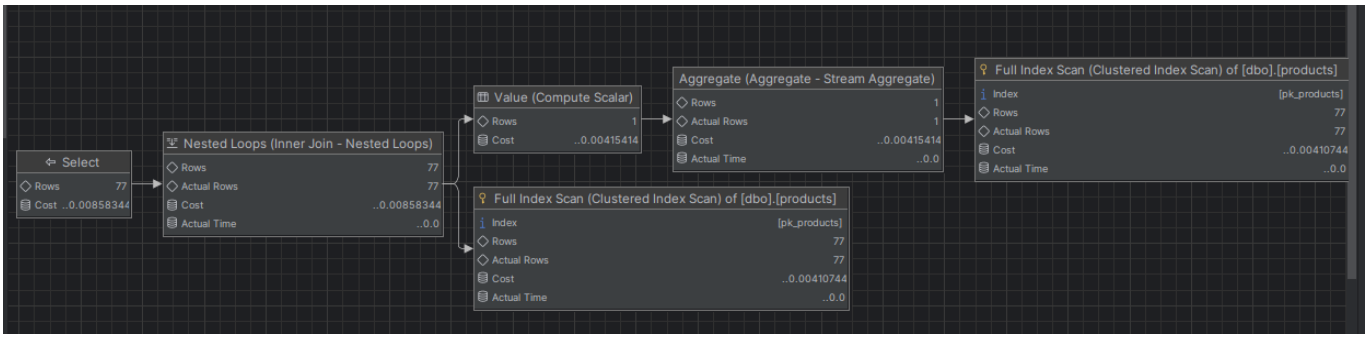
Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
Select		77		0.00859114		CardinalityEstimati...
Value (Compute Scalar)		77		0.00859114		AvgRowSize = 71;E...
Nested Loops (Inner Jc		77	77	0.00858344	1.0	AvgRowSize = 71;E...
Value (Compute Sca		1		0.00415414		AvgRowSize = 15;...
Aggregate (Aggrega		1	1	0.00415414	0.0	AvgRowSize = 15;...
Full Index Scar	table: [dbo].[products]; index: [pk_products];	77	77	0.00410744	0.0	AvgRowSize = 15;...
Full Index Scan (Clus	table: [dbo].[products]; index: [pk_products];	77	77	0.00410744	0.0	AvgRowSize = 63;...



Plan zawiera dwa Full Index Scan na tabeli products. Pierwszy skan odczytuje dane główne, drugi wykonuje agregację w ramach Compute Scalar, obliczając średnią. Wyniki łączone są przez Nested Loops (Inner Join), a na zewnątrz dodatkowy Compute Scalar dołącza wartość średniej do każdego wiersza. Podzapytanie jest więc materializowane jako osobna gałąź planu.

Output | Plan

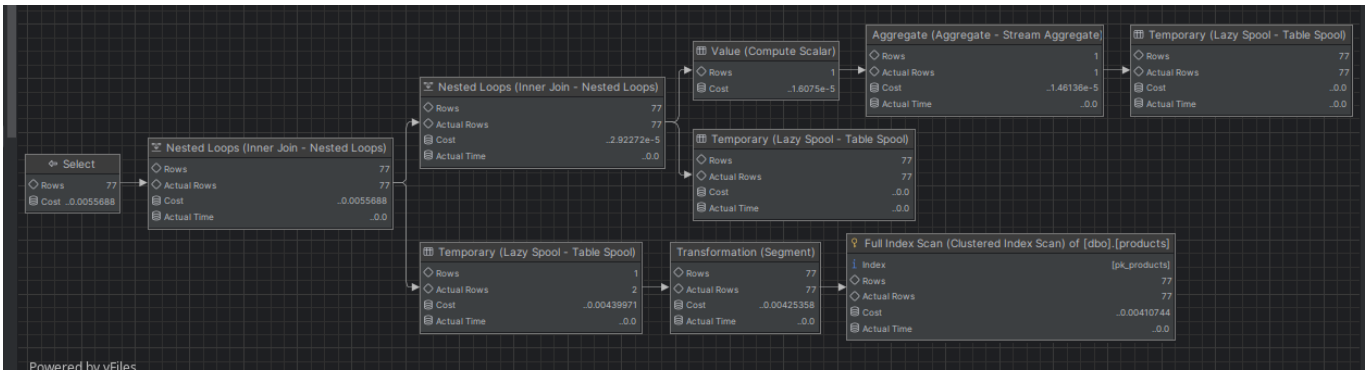
Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
Select		77		0.00858344		CardinalityEstimati...
Nested Loops (Inner Jo		77	77	0.00858344	0.0	AvgRowSize = 71;E...
Value (Compute Scal		1		0.00415414		AvgRowSize = 15;E...
Aggregate (Aggregat		1	1	0.00415414	0.0	AvgRowSize = 15;E...
Full Index Scan	table: [dbo].[products]; index: [pk_products];	77	77	0.00410744	0.0	AvgRowSize = 15;E...
Full Index Scan (Clus	table: [dbo].[products]; index: [pk_products];	77	77	0.00410744	0.0	AvgRowSize = 63;...



Plan jest podobny do powyższego: dwa Full Index Scan, Stream Aggregate wyliczający średnią, Compute Scalar oraz Nested Loops (Inner Join). SQL Server zoptymalizował oba zapytania do prawie tego samego planu. Różnicą jest brak jednego Compute Scalar, który nie musi występować, bo CROSS JOIN złączy średnią sam.

Output | Plan

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
Select		77		0.0055688		CardinalityEstimati...
Nested Loops (Inner Join		77	77	0.0055688	0.0	AvgRowSize = 71;E...
Temporary (Lazy Spool		1	2	0.00439971	0.0	AvgRowSize = 63;...
Transformation (Segme		77	77	0.00425358	0.0	AvgRowSize = 63;...
Full Index Scan ((	table: [dbo].[products]; index: [pk_products];	77	77	0.00410744	0.0	AvgRowSize = 63;...
Nested Loops (Inner Jc		77	77	2.92272E-5	0.0	AvgRowSize = 63;...
Value (Compute Sca		1		1.6075E-5		AvgRowSize = 63;...
Aggregate (Aggrega		1	1	1.46136E-5	0.0	AvgRowSize = 63;...
Temporary (La		77	77	0.0	0.0	AvgRowSize = 63;...
Temporary (Lazy Spo		77	77	0.0	0.0	AvgRowSize = 63;...



Jeden Full Index Scan odczytuje dane, następnie Transformation (Segment) dzieli wiersze na partycje (tutaj: jedna partycja, to cała tabela), Temporary (Lazy Spool) buforuje dane, a Stream Aggregate wylicza średnią raz z bufora. Wyniki łączone są przez dwa Nested Loops.

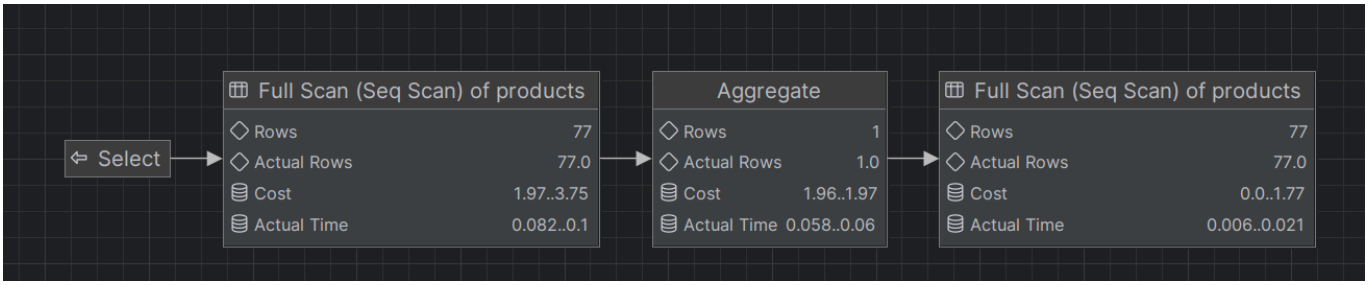
Fumkcja okna ma najszybszy czas wykonania, a pozostałe dwa plany są bardzo podobne do siebie przez co też czas wykonania jets prawie identyczny.

ostgreSql

Session | Result 66

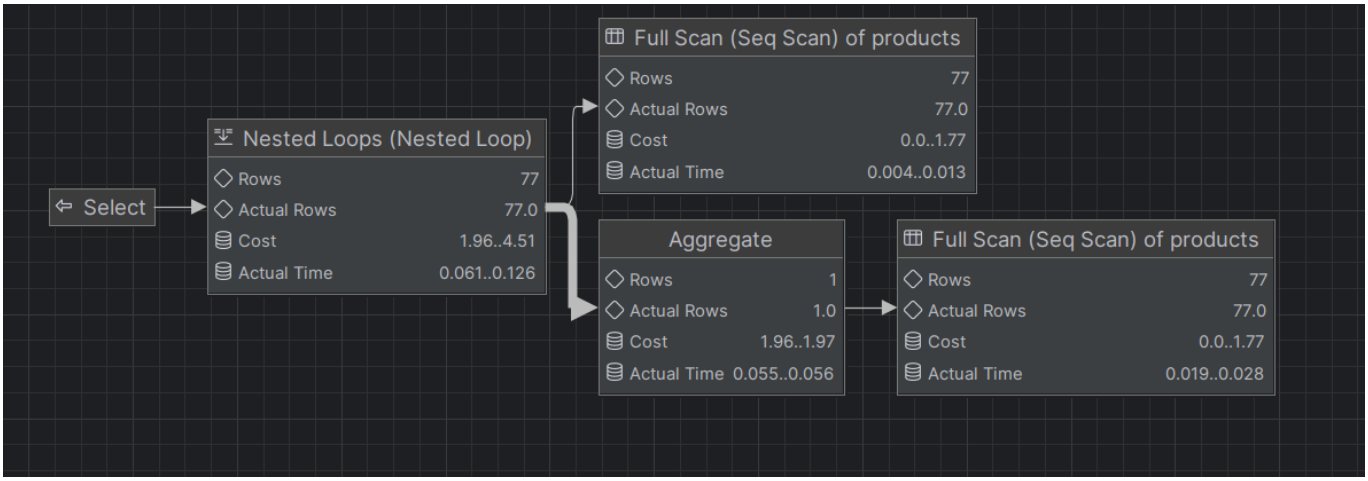
Output | Plan

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
Select								Planning = {"Sha...
Full Scan (Seq Sca	table: products;	77	77.0	3.75	0.1	1.97	0.082	Parallel Aware = ...
Aggregate		1	1.0	1.97	0.06	1.96	0.058	Strategy = Plain;...
Full Scan (Se	table: products;	77	77.0	1.77	0.021	0.0	0.006	Parent Relations...



Plan zawiera dwa Full Scan of products. Jeden skan dla zapytania głównego, drugi dla podzapytania wewnątrz Aggregate. Oba potem są łączone.

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
Select								Planning = {"Sha...
Nested Loops (Ne		77	77.0	4.51	0.126	1.96	0.061	Parallel Aware = ...
Aggregate		1	1.0	1.97	0.056	1.96	0.055	Strategy = Plain;...
Full Scan (Se	table: products;	77	77.0	1.77	0.028	0.0	0.019	Parent Relations...
Full Scan (Seq S	table: products;	77	77.0	1.77	0.013	0.0	0.004	Parent Relations...



Podobnie jak wyżej: dwa Full Scan of products, jeden pod Aggregate, złączone przez Nested Loop (zagnieżdżone pętle).

Session | Result 67

Output | Plan

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
Select								Planning = {"Shar...
Transformation (Wi		77	77.0	2.73	0.133	2.7	0.107	Parallel Aware = ...
Full Scan (Se	table: products;	77	77.0	1.77	0.046	0.0	0.027	Parent Relations...

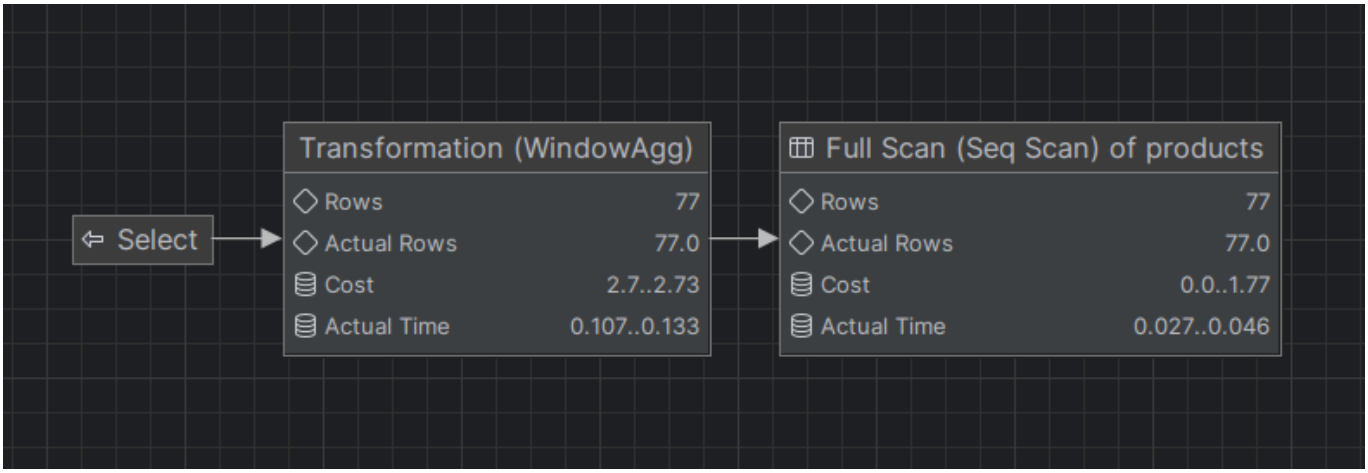
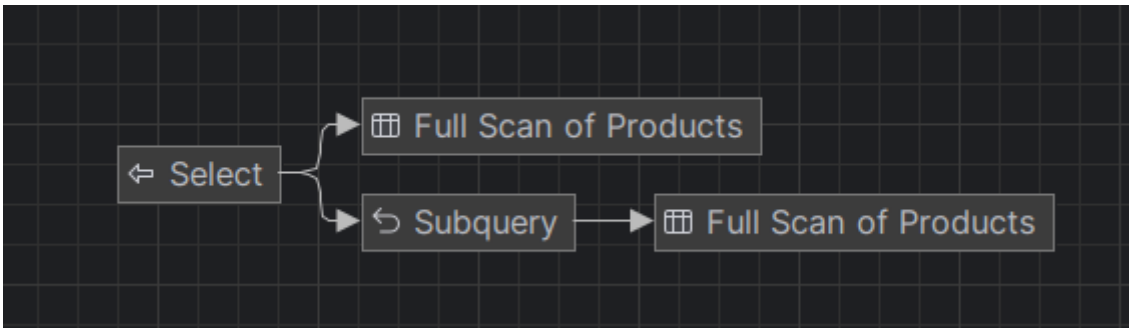


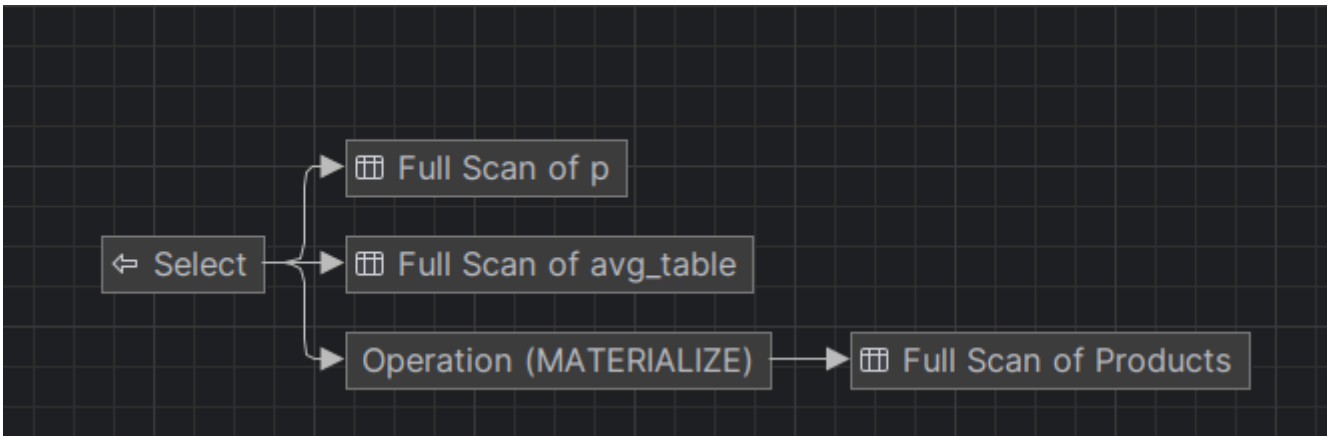
Tabela skanowana tylko raz - WindowAgg oblicza średnią w jednym przebiegu.

Wszystkie wersje mają bardzo zbliżony do siebie czas wykonania.

SQLite

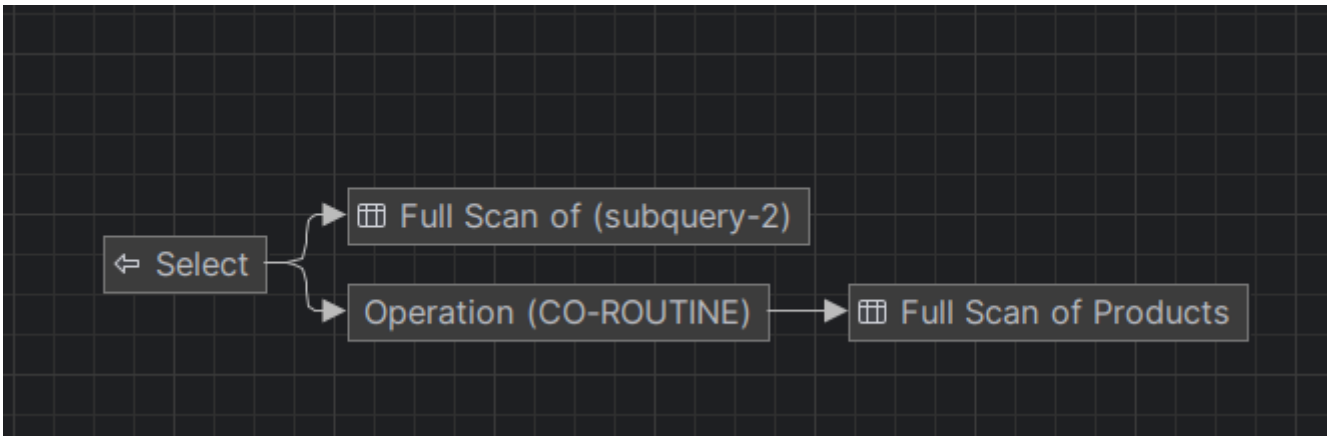


Plan zawiera dwa odczyty tabeli - Full Scan of Products dla zapytania głównego i podzapytania, potem je łączy do jednej tabeli.



Plan zawiera trzy gałęzie - SQLite materializuje podzapytanie jako tabele tymczasową po przeskanowaniu tabeli Products, odczytuje ją (tymczasową tabele) i znowu wykonuje Full Scan of Products i dokleja wynik z

avg_table.



SQLite implementuje funkcję okna jako coroutine - jeden odczyt danych, który oblicza średnią. Zapytanie główne czyta wyniki obliczane przez funkcję okna i je wykorzystuje.

SQLite nie podaje kosztów zapytań.

Zadanie 4

Baza: Northwind, tabela products

Napisz polecenie, które zwraca: id produktu, nazwę produktu, cenę produktu, średnią cenę produktów w kategorii, do której należy dany produkt. Wyświetl tylko pozycje (produkty) których cena jest większa niż średnia cena.

Napisz polecenie z wykorzystaniem podzapytania, join'a oraz funkcji okna. Porównaj zapytania. Porównaj czasy oraz plany wykonania zapytań.

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSql, SQLite)

> Wyniki:

Podzapytanie

```
select p1.productid, p1.productname, p1.unitprice,
       (select avg(unitprice) from products p2 where p1.categoryid =
p2.categoryid) as avgprice
from products p1
where p1.unitprice > (select avg(unitprice) from products p2 where p1.categoryid =
p2.categoryid)
```

Join

```sql
select p.productid, p.productname, p.unitprice, c.avgprice
from products p
join (
    select categoryid, avg(unitprice) as avgprice
    from products
    group by categoryid
) c on p.categoryid = c.categoryid
where p.unitprice > c.avgprice
```

Funkcja okna

```sql
with avg as (
    select productid, productname, unitprice,
           avg(unitprice) over (partition by categoryid) as avgprice
    from products
)
select productid, productname, unitprice, avgprice
from avg
where unitprice > avgprice
```

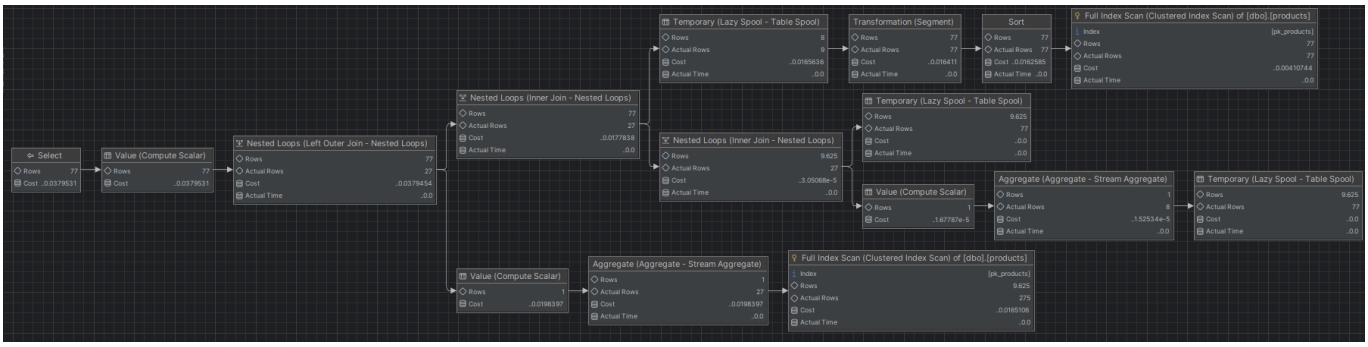
MS SQL Server

Podzapytanie

Output | Plan

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
Select		77		0.0379531		CardinalityEstimationMo...
Value (Compute Scalar)		77		0.0379531		AvgRowSize = 71;Estima...
Nested Loops (Left Outer Join - N		77	27	0.0379454	0.0	AvgRowSize = 71;Estima...
Nested Loops (Inner Join - Nes		77	27	0.0177838	0.0	AvgRowSize = 67;Estima...
Temporary (Lazy Spool - Tal		8	9	0.0165636	0.0	AvgRowSize = 67;Estima...
Transformation (Segment)		77	77	0.016411	0.0	AvgRowSize = 67;Estima...
Sort		77	77	0.0162585	0.0	AvgRowSize = 67;Estima...
Full Index Scan (Cl	table: [dbo].[products]; index: [pk_products];	77	77	0.00410744	0.0	AvgRowSize = 67;Estima...
Nested Loops (Inner Join - I		9.625	27	3.05068E-5	0.0	AvgRowSize = 67;Estima...
Value (Compute Scalar)		1		1.67787E-5		AvgRowSize = 67;Estima...
Aggregate (Aggregate -		1	8	1.52534E-5	0.0	AvgRowSize = 67;Estima...
Temporary (Lazy Sp		9.625	77	0.0	0.0	AvgRowSize = 67;Estima...
Temporary (Lazy Spool -		9.625	77	0.0	0.0	AvgRowSize = 67;Estima...
Value (Compute Scalar)		1		0.0198397		AvgRowSize = 15;Estima...
Aggregate (Aggregate - Strean		1	27	0.0198397	0.0	AvgRowSize = 15;Estima...
Full Index Scan (Clustere	table: [dbo].[products]; index: [pk_products];	9.625	275	0.0165106	0.0	AvgRowSize = 19;Estima...

Operations Tree

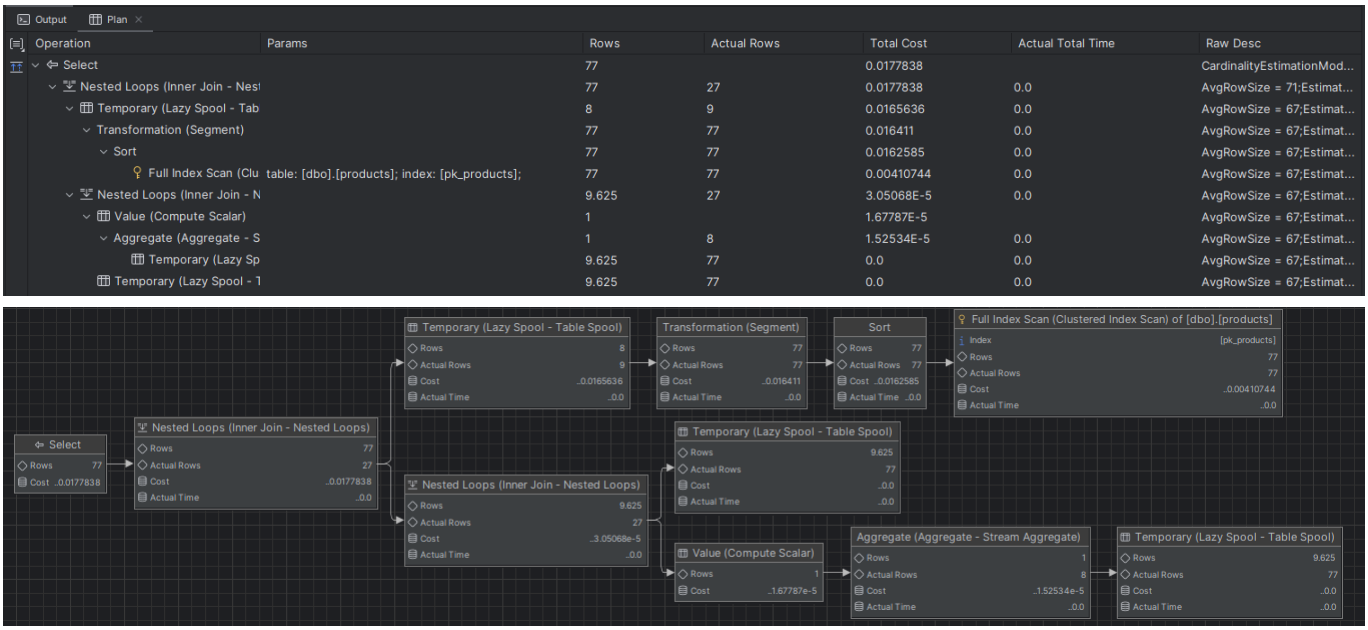


Koszt: 0.038

Czas: 0.0ms

Plan wykonania dla tego zapytania jest złożony i stosunkowo kosztowny. Wymusza on np. podwójny odczyt tabeli products `Full Index Scan` – dla zapytania głównego i podzapytania. Oznacza to, że silnik bazy danych musiał sięgnąć do tabeli `products` dwa razy, raz żeby pobrać listę produktów i drugi raz, żeby zebrać dane do wyliczenia średnich. Optymalizator częściowo ratuje wydajność, buforując wyliczone średnie dla każdej kategorii w pamięci `Lazy Spool`, by nie liczyć ich wielokrotnie. Czas wykonania to 0.0 ms wyłącznie dzięki małemu rozmiarowi tabeli, wynoszącemu 77 wierszy. Przy dużej ilości danych, duża liczba operacji Nested Loops zapewne drastycznie obniżyłaby wydajność tego zapytania.

Join



Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
Select		77		0.0177838		CardinalityEstimationMod...
Nested Loops (Inner Join - Nest		77	27	0.0177838	0.0	AvgRowSize = 71;Estimat...
Temporary (Lazy Spool - Tab		8	9	0.0165636	0.0	AvgRowSize = 67;Estimat...
Transformation (Segment)		77	77	0.016411	0.0	AvgRowSize = 67;Estimat...
Sort		77	77	0.0162585	0.0	AvgRowSize = 67;Estimat...
Full Index Scan (Clu	table: [dbo].[products]; index: [pk_products];	77	77	0.00410744	0.0	AvgRowSize = 67;Estimat...
Nested Loops (Inner Join - N		9.625	27	3.05068E-5	0.0	AvgRowSize = 67;Estimat...
Value (Compute Scalar)		1		1.67787E-5		AvgRowSize = 67;Estimat...
Aggregate (Aggregate - S		1	8	1.52534E-5	0.0	AvgRowSize = 67;Estimat...
Temporary (Lazy Sp		9.625	77	0.0	0.0	AvgRowSize = 67;Estimat...
Temporary (Lazy Spool - T		9.625	77	0.0	0.0	AvgRowSize = 67;Estimat...

Koszt: 0.018

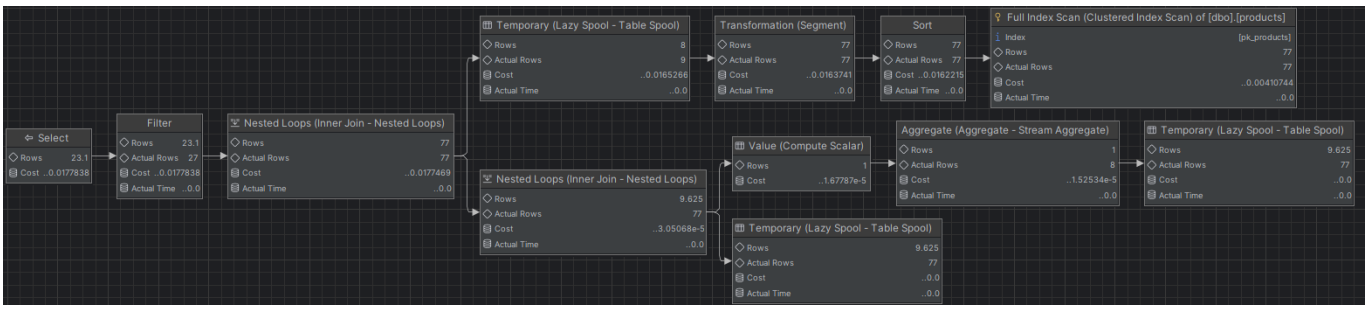
Czas: 0.0ms

Plan wykonania dla tego zapytania jest zauważalnie prostszy. Koszt spadł dwuktornie w porównaniu do wersji z podzapytaniem. Najważniejszą różnicą jest to, że silnik bazy danych wykonał tylko jeden odczyt tabeli z dysku `Full Index Scan`. Optymalizator odczytał tabelę products tylko raz, posortował dane i umieścił je w buforze pamięci tymczasowej `Lazy Spool`. Następnie wykorzystał ten bufor do wyliczenia średnich i złączenia danych `Nested Loops`. Czas wykonania to wciąż 0.0 ms ze względu na 77 wierszy w tabeli, jednak wyeliminowanie podwójnego fizycznego skanowania tabeli sprawia, że ta metoda jest bez porównania wydajniejsza i o wiele lepiej poradziłaby sobie przy dużej ilości danych.

Funkcja okna

Output | Plan

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
Select		23.1		0.0177838		CardinalityEstimationMod...
Filter		23.1	27	0.0177838	0.0	AvgRowSize = 71;Estimat...
Nested Loops (Inner Join - Nes		77	77	0.0177469	0.0	AvgRowSize = 71;Estimat...
Temporary (Lazy Spool - Tab		8	9	0.0165266	0.0	AvgRowSize = 67;Estimat...
Transformation (Segment)		77	77	0.0163741	0.0	AvgRowSize = 67;Estimat...
Sort		77	77	0.0162215	0.0	AvgRowSize = 67;Estimat...
Full Index Scan (Clu	table: [dbo].[products]; index: [pk_products];	77	77	0.00410744	0.0	AvgRowSize = 67;Estimat...
Nested Loops (Inner Join - N		9.625	77	3.05068E-5	0.0	AvgRowSize = 67;Estimat...
Value (Compute Scalar)		1		1.67787E-5		AvgRowSize = 67;Estimat...
Aggregate (Aggregate - S		1	8	1.52534E-5	0.0	AvgRowSize = 67;Estimat...
Temporary (Lazy Sp		9.625	77	0.0	0.0	AvgRowSize = 67;Estimat...
Temporary (Lazy Spool -		9.625	77	0.0	0.0	AvgRowSize = 67;Estimat...



Koszt: 0.018

Czas: 0.0ms

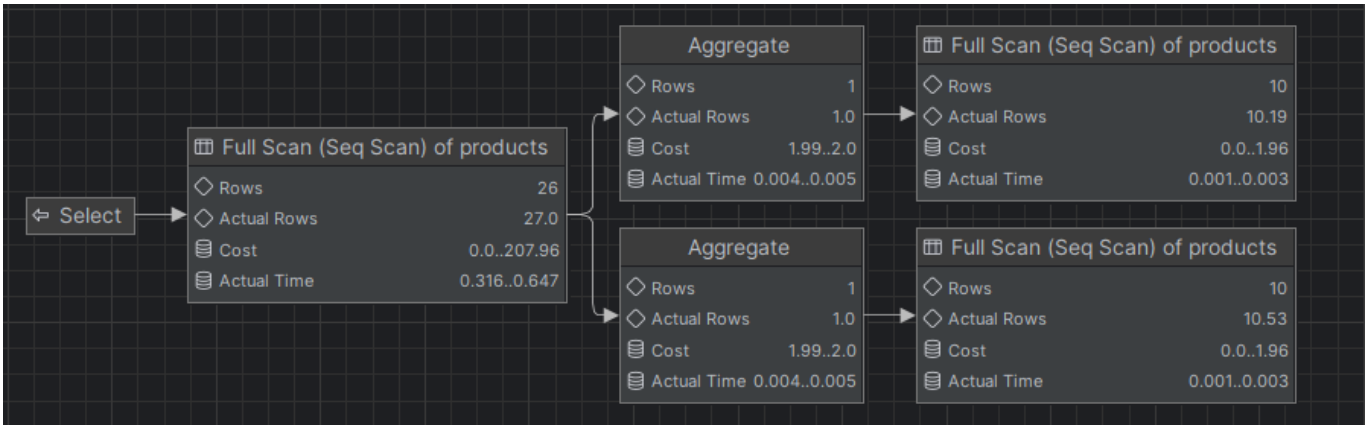
To podejście okazało się tak samo wydajne jak `JOIN`, plany wykonania wyglądają prawie identycznie. Z planu wykonania wynika, że baza skanuje tabelę products tylko jeden raz, co jest dużą oszczędnością w porównaniu do podzapytania. Baza dzieli dane na segmenty według kategorii, a następnie, zamiast zwijać wiersze jak przy zwykłym grupowaniu, wylicza średnią raz dla każdej kategorii i dopisuje ją do każdego produktu. Na samym końcu nakładany jest filtr, który odrzuca produkty tańsze niż średnia. Dzięki zastosowaniu konstrukcji `WITH`, mamy pewność, że filtr zadziała na już przeliczonych danych, co widać po strukturze drzewa operacji.

PostgreSql

Podzapytanie

Output | Plan

Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
Select								Planning = {"Shared Hit...
Full Scan (Seq Scan)	table: products;	26	27.0	207.96	0.647	0.0	0.316	Parallel Aware = false;...
Aggregate		1	1.0	2.0	0.005	1.99	0.004	Strategy = Plain;Partial ...
Full Scan (Seq Sc	table: products;	10	10.19	1.96	0.003	0.0	0.001	Parent Relationship = ...
Aggregate		1	1.0	2.0	0.005	1.99	0.004	Strategy = Plain;Partial ...
Full Scan (Seq Sc	table: products;	10	10.53	1.96	0.003	0.0	0.001	Parent Relationship = ...

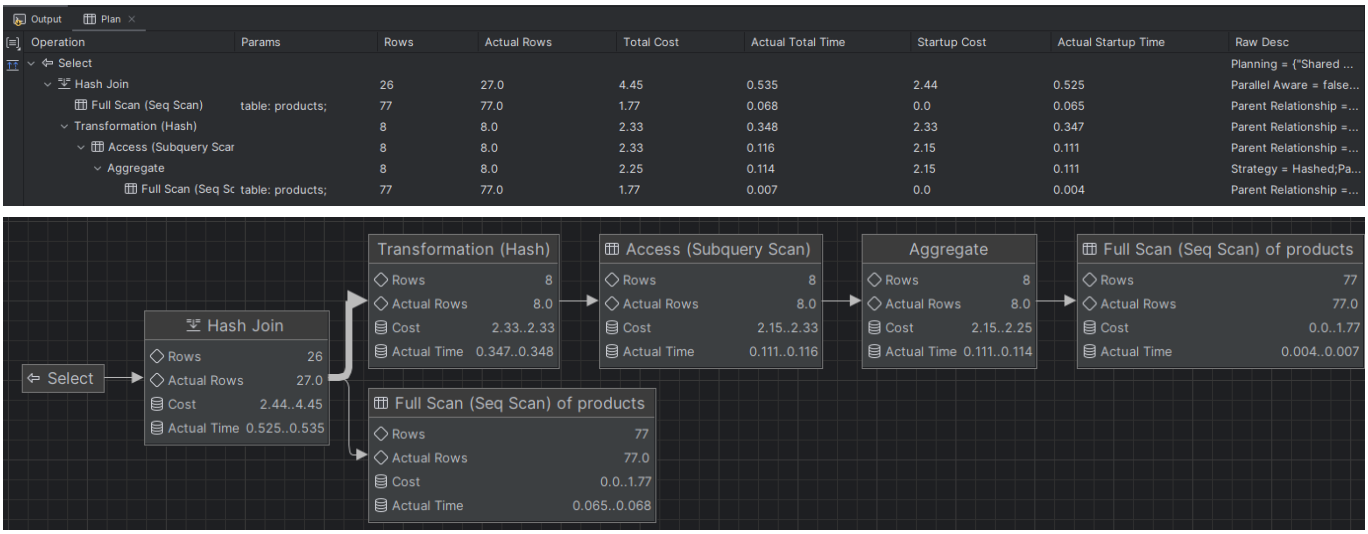


Koszt: 207.96

Czas: 0.647ms

Z planu wykonania wynika, że to zapytanie jest bardzo nieoptymalne. Główny węzeł wykonuje pełne skanowanie tabeli wiersz po wierszu `Full Scan (Seq Scan)`. Ponieważ podzapytanie zostało użyte w kodzie dwukrotnie (raz w `SELECT`, drugi raz w `WHERE`), diagram rozgałęzia się na dwa osobne tory obliczeniowe, co prowadzi do tego, że dla każdego z 77 odczytanych produktów najpierw od nowa przeszukuje tabelę i liczy średnią dla jego kategorii `Aggregate`, aby sprawdzić warunek z klauzuli `WHERE`. Następnie, jeśli wiersz spełnia warunek, baza zamiast zapamiętać ten wynik, uruchamia drugi tor i liczy dokładnie tę samą średnią od nowa, by wyświetlić ją w `SELECT`. Stąd bierze się tak wysoki koszt całkowity. Choć przy małej tabeli czas wykonania to zaledwie 0.647ms, tego typu struktura zapytania drastycznie obniżyłaby wydajność przy pracy na dużej ilości danych.

Join



Output | Plan

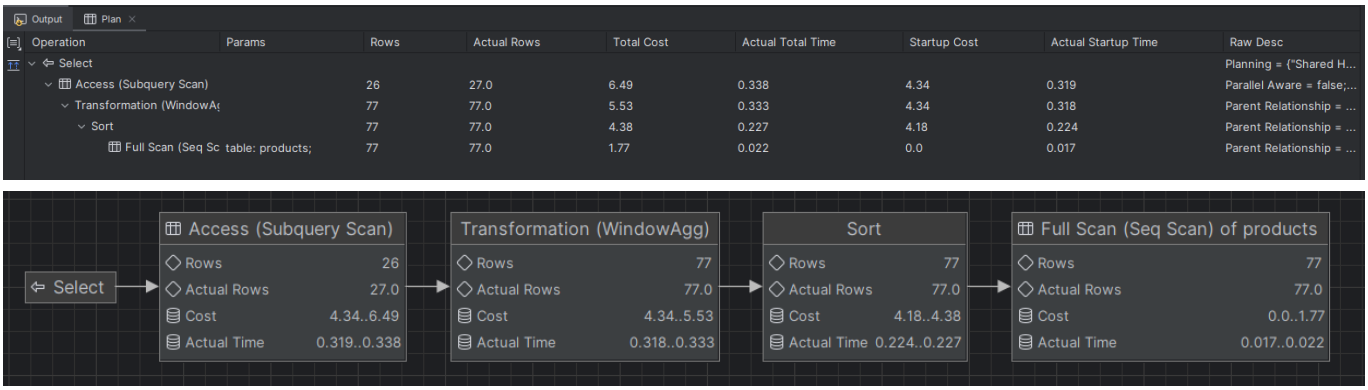
Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
Select								Planning = {"Shared ...
Hash Join		26	27.0	4.45	0.535	2.44	0.525	Parallel Aware = false...
Full Scan (Seq Scan)	table: products;	77	77.0	1.77	0.068	0.0	0.065	Parent Relationship =...
Transformation (Hash)		8	8.0	2.33	0.348	2.33	0.347	Parent Relationship =...
Access (Subquery Scar		8	8.0	2.33	0.116	2.15	0.111	Parent Relationship =...
Aggregate		8	8.0	2.25	0.114	2.15	0.111	Strategy = Hashed;Pa...
Full Scan (Seq Sc	table: products;	77	77.0	1.77	0.007	0.0	0.004	Parent Relationship =...

Koszt: 4.45

Czas: 0.535ms

Zastosowanie złączenia `JOIN` z grupowaniem zoptymalizowało zapytanie, redukując koszt całkowity z 207.96 do zaledwie 4.45. Z planu wykonania wynika, operacja dzieli się na dwa etapy. Najpierw baza jednokrotnie wylicza średnie dla poszczególnych kategorii `Aggregate` i umieszcza je w pamięci operacyjnej `Transformation (Hash)`. Następnie wykonuje pojedynczy odczyt wszystkich produktów i łączy je z obliczonymi wcześniej średnimi za pomocą operatora `Hash Join`. Takie podejście eliminuje problem wielokrotnego skanowania tabeli i powtarzania tych samych obliczeń, co czyni zapytanie wydajnym.

Funkcja okna



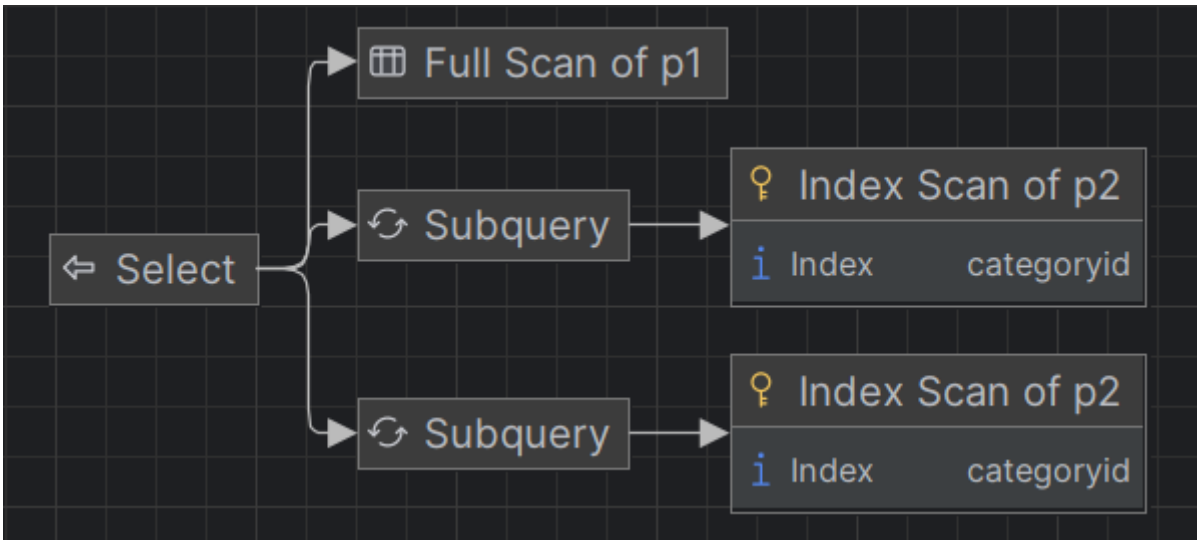
Koszt: 6.49

Czas: 0.338ms

Plan wykonania dla funkcji okna ma strukturę w pełni liniową, bez żadnych rozgałęzień. Baza danych wykonuje skanowanie tabeli `Seq Scan`, a następnie sortuje dane według ID kategorii `Sort`. Na tak posortowanym zbiorze uruchamiany jest operator `WindowAgg`, który wylicza średnie dla poszczególnych kategorii. Cały, przeliczony zestaw danych przekazywany jest wyżej `Subquery Scan`, gdzie następuje końcowe filtrowanie odrzucające produkty poniżej średniej. Choć koszt całkowity `6.49` jest minimalnie wyższy niż w przypadku złączenia `4.45`, rozwiązanie to pozostaje wysoce optymalne, eliminując konieczność wielokrotnego skanowania tabeli.

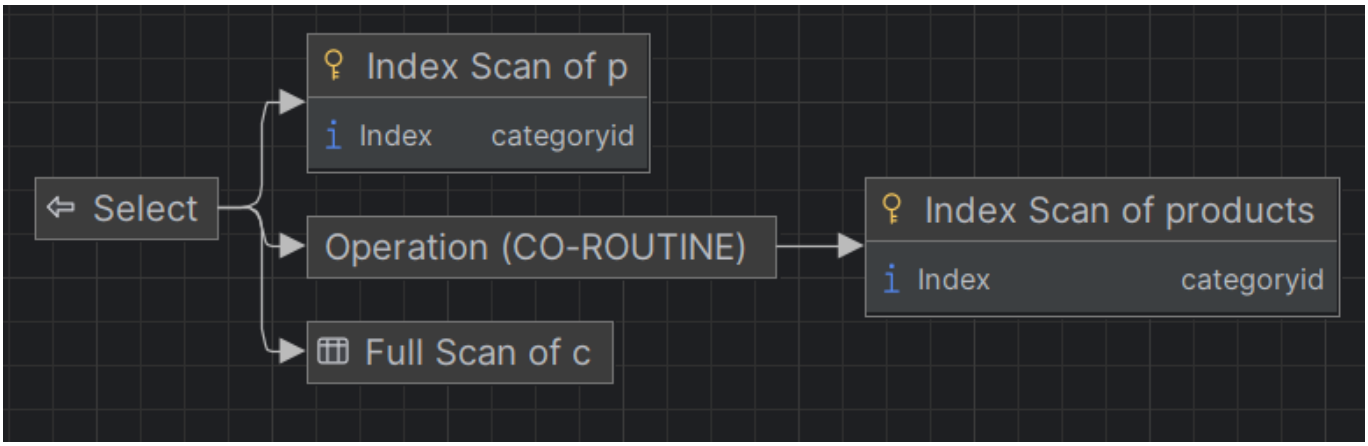
SQLite

Podzapytanie



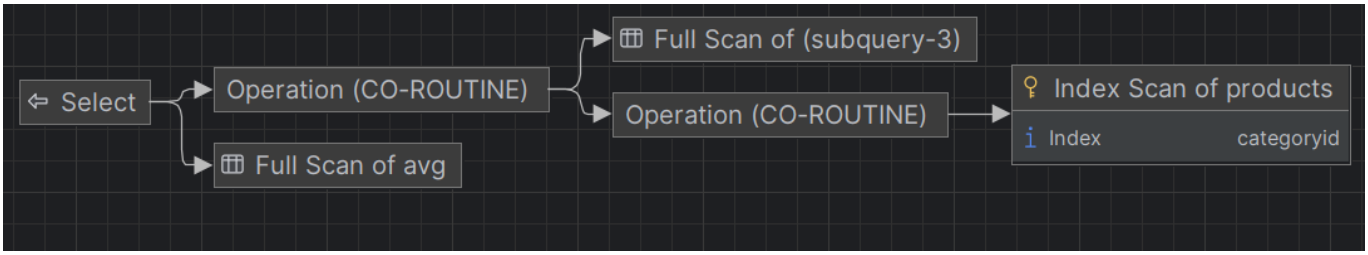
Z planu wynika, że zapytanie jest nieoptymalne. Baza wykonuje pełne skanowanie tabeli głównej `Full Scan of p1`, a następnie dla każdego odczytanego wiersza uruchamia dwa oddzielne podzapytania `Subquery`. Chociaż SQLite optymalizuje te podzapytania używając indeksu `Index Scan of p2 na categoryid`, to konieczność ciągłego powtarzania tych samych obliczeń sprawia, że rozwiązanie to jest bardzo niewydajne.

Join



Z planu wynika, że zastosowanie złączenia `JOIN` znacząco optymalizuje działanie bazy. SQLite wydziela obliczanie średnich do operacji `Operation (CO-ROUTINE)`, grupując dane przy pomocy indeksu `Index Scan of products`. Wynik tych obliczeń traktowany jest jako tymczasowy zbiór `Full Scan of c`, który następnie jest łączony z główną tabelą produktów, co również wspiera indeks `Index Scan of p`. Dzięki temu baza liczy średnie tylko raz dla każdej kategorii.

Funkcja okna



Z planu wykonania wynika, że SQLite radzi sobie z funkcją okna bardzo sprawnie. Najpierw baza wykorzystuje indeks `Index Scan of products`, aby szybko pogrupować produkty według kategorii i policzyć dla nich średnie. Cały ten przeliczony zbiór danych (pochodzący z klauzuli WITH) jest tymczasowo zapisywany w pamięci jako wirtualna tabela `avg`. W drugim kroku zapytanie główne wykonuje już tylko jeden, prosty odczyt tej gotowej tabeli `Full Scan of avg` odrzucając te produkty, których cena jest niższa lub równa wyliczonej średniej.

Zadanie 5

Oryginalna baza Northwind jest bardzo mała. Warto zaobserwować działanie na nieco większym zbiorze danych.

Baza Northwind3 zawiera dodatkową tabelę product_history

- 2,2 mln wierszy

Bazę Northwind3 można pobrać z moodle (zakładka - Backupy baz danych)

Można też wygenerować tabelę product_history przy pomocy skryptu

Skrypt dla SQL Srerver

Stwórz tabelę o następującej strukturze:

```
create table product_history(
   id int identity(1,1) not null,
   productid int,
   productname varchar(40) not null,
   supplierid int null,
   categoryid int null,
   quantityperunit varchar(20) null,
   unitprice decimal(10,2) null,
   quantity int,
   value decimal(10,2),
   date date,
 constraint pk_product_history primary key clustered
    (id asc )
)
```

Wygeneruj przykładowe dane:

Dla 30000 iteracji, tabela będzie zawierała nieco ponad 2mln wierszy (dostostu ograniczenie do możliwości swojego komputera)

Skrypt dla SQL Srerver

```
declare @i int
set @i = 1
while @i <= 30000
begin
    insert product_history
    select productid, ProductName, SupplierID, CategoryID,
         QuantityPerUnit,round(RAND()*unitprice + 10,2),
         cast(RAND() * productid + 10 as int), 0,
         dateadd(day, @i, '1940-01-01')
    from products
    set @i = @i + 1;
end;

update product_history
set value = unitprice * quantity
where 1=1;
```

Skrypt dla Postgresql

```
create table product_history(
   id int generated always as identity not null
       constraint pkproduct_history
            primary key,
   productid int,
```

```
    productname varchar(40) not null,
    supplierid int null,
    categoryid int null,
    quantityperunit varchar(20) null,
    unitprice decimal(10,2) null,
    quantity int,
    value decimal(10,2),
    date date
);
```

Wygeneruj przykładowe dane:

Skrypt dla Postgresql

```
do $$
begin
  for cnt in 1..30000 loop
    insert into product_history(productid, productname, supplierid,
           categoryid, quantityperunit,
           unitprice, quantity, value, date)
    select productid, productname, supplierid, categoryid,
           quantityperunit,
           round((random()*unitprice + 10)::numeric,2),
           cast(random() * productid + 10 as int), 0,
           cast('1940-01-01' as date) + cnt
    from products;
  end loop;
end; $$;

update product_history
set value = unitprice * quantity
where 1=1;
```

Wykonaj polecenia: `select count(*) from product_history`, potwierdzające wykonanie zadania

> Wyniki:

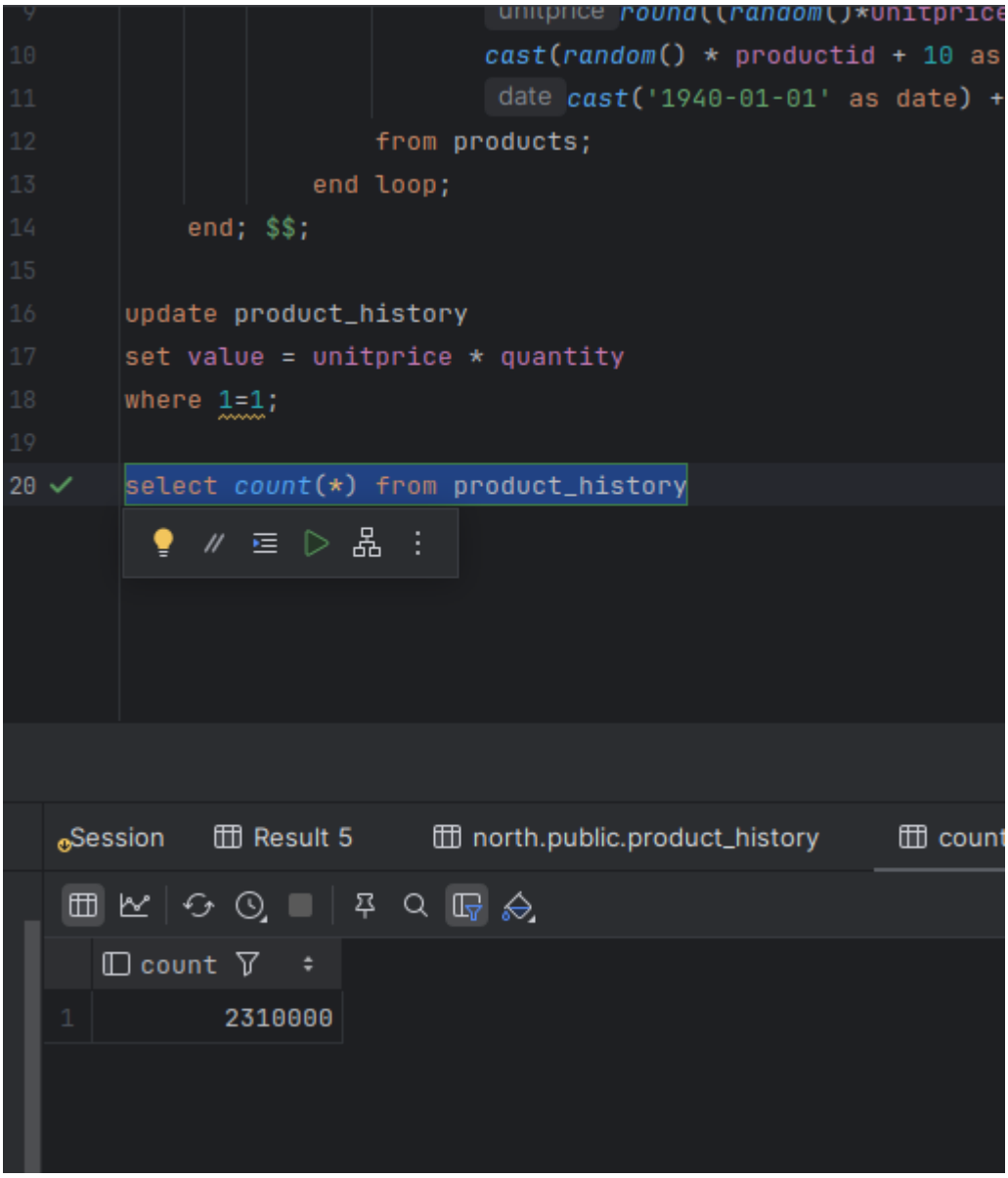
SQL Srerver

```sql
declare @i int
set @i = 1
while @i <= 30000
    begin
        insert product_history
        select id productid, productid ProductName, productname SupplierI
               categoryid QuantityPerUnit, quantityperunit round(RAND()*unitp
               cast(RAND() * productid + 10 as int), quantity 0,
               value dateadd(day, @i, '1940-01-01')
        from products
        set @i = @i + 1;
    end;

update product_history
set value = unitprice * quantity
where 1=1;


select count(*) from product_history
```

Session | count(*):int 2 | north.public.product_history | count(*):bigint

	<anonymous>
1	2310000

Postgresql



O wiele dłużej polecenie wykonywało się w sql serwerze (pare minut) niż w postgresie (pare sekund).

Zadanie 6

Baza: Northwind, tabela product_history

Napisz polecenie, które zwraca: id pozycji, id produktu, nazwę produktu, id_kategorii, cenę produktu, średnią cenę produktów w kategorii do której należy dany produkt. Wyświetl tylko pozycje (produkty) których cena jest większa niż średnia cena.

W przypadku długiego czasu wykonania ogranicz zbiór wynikowy do kilkuset/kilku tysięcy wierszy

pomocna może być konstrukcja `with`

```
with t as (

....
)
```

```
select * from t
where id between ....
```

Napisz polecenie z wykorzystaniem podzapytania, join'a oraz funkcji okna. Porównaj zapytania. Porównaj czasy oraz plany wykonania zapytań.

Przetestuj działanie w różnych SZBD (MS SQL Server, PostgreSql, SQLite)

> Wyniki:

Zgodnie z poleceniem ograniczono analizowany zbiór do 10 000 wierszy za pomocą klauzuli `WITH`.

Podzapytanie

```
with t as (select * from product_history where id between 1 and 10000)
select t1.id,
       t1.productid,
       t1.productname,
       t1.categoryid,
       t1.unitprice,
       (select avg(unitprice) from t t2 where t1.categoryid = t2.categoryid) as
avgprice
from t t1
where t1.unitprice > (select avg(unitprice) from t t2 where t1.categoryid =
t2.categoryid)
```

Join

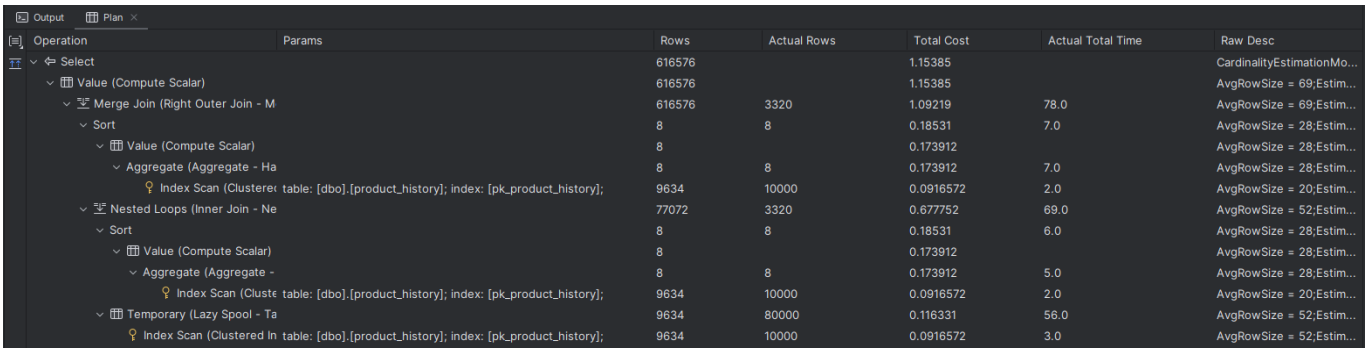
```
with t as (select * from product_history where id between 1 and 10000)
select t.id,
       t.productid,
       t.productname,
       t.categoryid,
       t.unitprice,
       c.avgprice
from t
         join (select categoryid, avg(unitprice) as avgprice
               from t
               group by categoryid) as c on t.categoryid = c.categoryid
where t.unitprice > c.avgprice
```

Funkcja okna

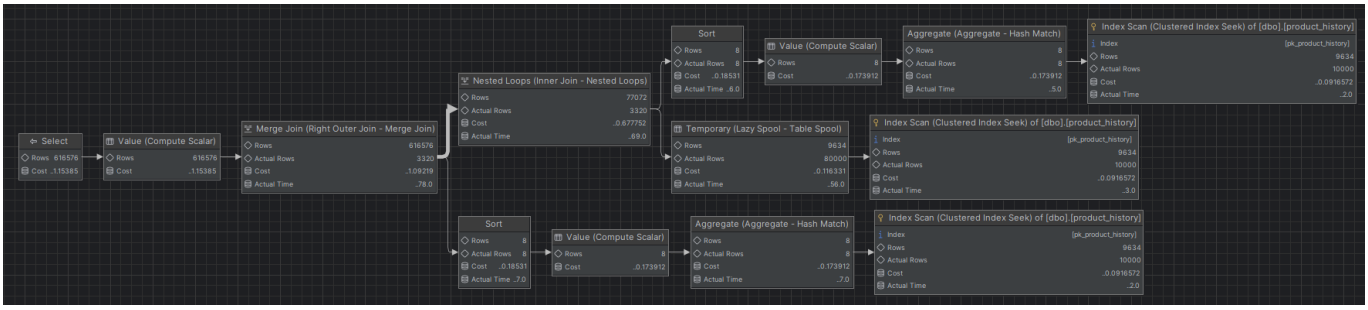
```sql
with t as (select * from product_history where id between 1 and 10000),
     t_avg as (select id,
                      productid,
                      productname,
                      categoryid,
                      unitprice,
                      avg(unitprice) over (partition by categoryid) as avgprice
               from t)
select id,
       productid,
       productname,
       categoryid,
       unitprice,
       avgprice
from t_avg
where unitprice > avgprice
```

MS SQL Server

Podzapytanie



Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Raw Desc
Select		616576		1.15385		CardinalityEstimationMo...
Value (Compute Scalar)		616576		1.15385		AvgRowSize = 69;Estim...
Merge Join (Right Outer Join - M		616576	3320	1.09219	78.0	AvgRowSize = 69;Estim...
Sort		8	8	0.18531	7.0	AvgRowSize = 28;Estim...
Value (Compute Scalar)		8		0.173912		AvgRowSize = 28;Estim...
Aggregate (Aggregate - Ha		8	8	0.173912	7.0	AvgRowSize = 28;Estim...
Index Scan (Clustere	table: [dbo].[product_history]; index: [pk_product_history];	9634	10000	0.0916572	2.0	AvgRowSize = 20;Estim...
Nested Loops (Inner Join - Ne		77072	3320	0.677752	69.0	AvgRowSize = 52;Estim...
Sort		8	8	0.18531	6.0	AvgRowSize = 28;Estim...
Value (Compute Scalar)		8		0.173912		AvgRowSize = 28;Estim...
Aggregate (Aggregate -		8	8	0.173912	5.0	AvgRowSize = 28;Estim...
Index Scan (Cluste	table: [dbo].[product_history]; index: [pk_product_history];	9634	10000	0.0916572	2.0	AvgRowSize = 20;Estim...
Temporary (Lazy Spool - Ta		9634	80000	0.116331	56.0	AvgRowSize = 52;Estim...
Index Scan (Clustered In	table: [dbo].[product_history]; index: [pk_product_history];	9634	10000	0.0916572	3.0	AvgRowSize = 52;Estim...

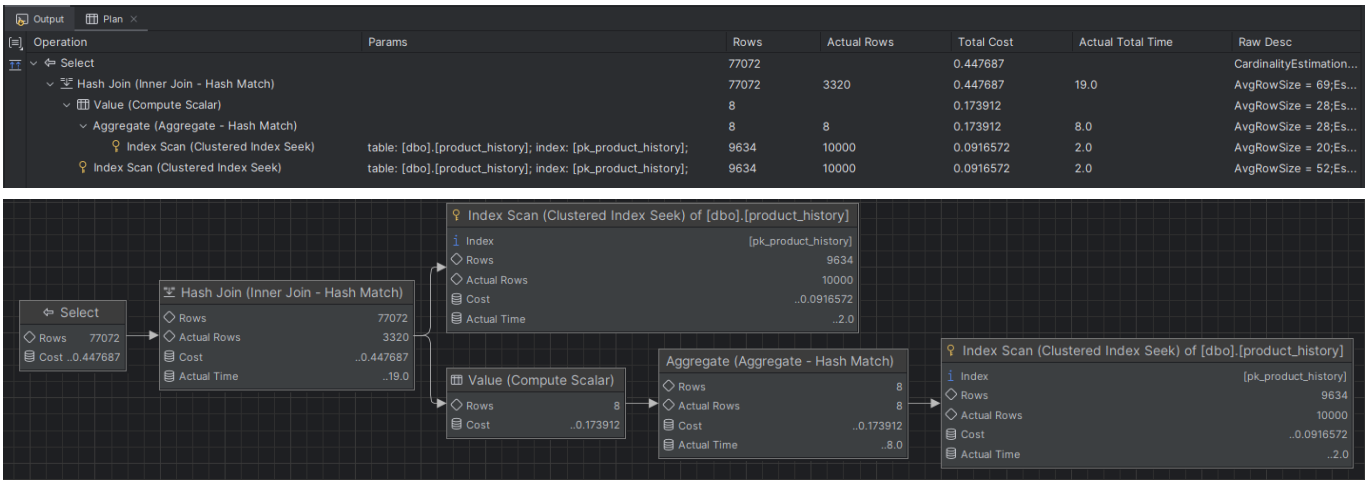


Koszt: 1.154

Czas: 78ms

Na planie wykonania widoczny jest trzykrotny odczyt tabeli `Index Scan`. Baza skanuje dane raz dla głównego zapytania oraz dwa razy osobno dla podzapytań (dla klauzuli SELECT i WHERE). Średnie są liczone dwukrotnie w dwóch osobnych gałęziach planu. Koszt wynoszący 1.154 i czas 77 ms są stosunkowo wysokie. Wynika to z faktu, że silnik musi wykonywać te same obliczenia wielokrotnie.

Join

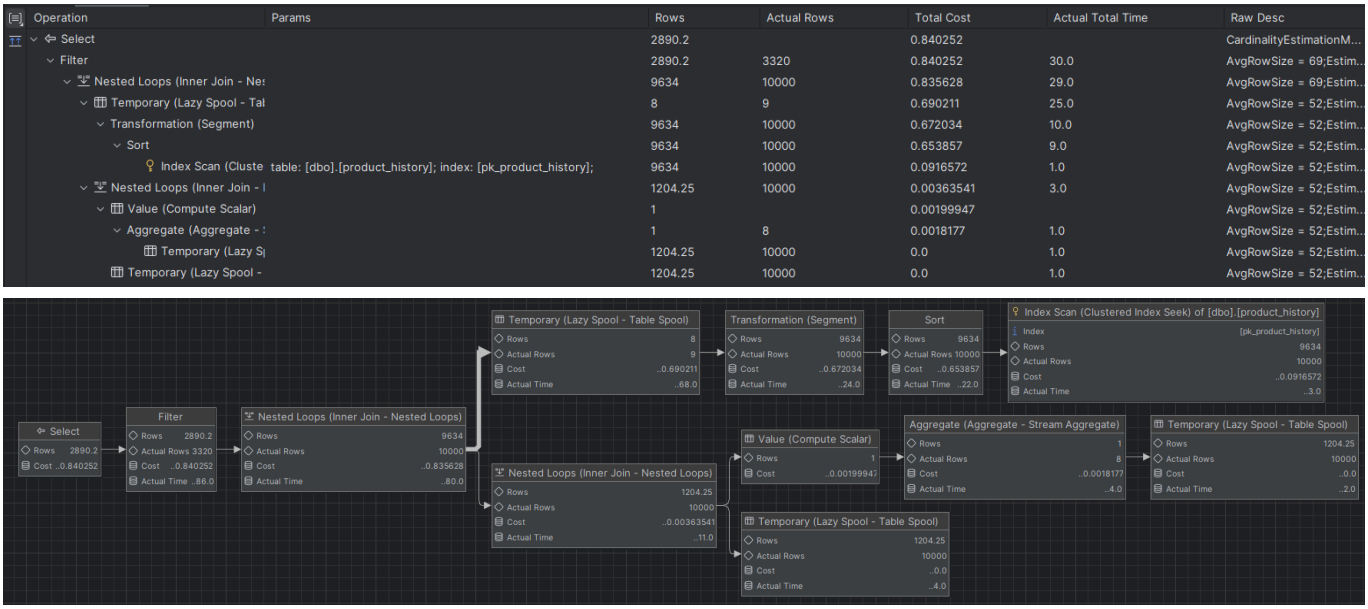


Koszt: 0.448

Czas: 19ms

Zastosowanie złączenia `JOIN` znacząco zoptymalizowało zapytanie. Czas wykonania spadł z 78 ms do 19 ms, a koszt z 1.154 do 0.448. Na planie wykonania widać tylko dwukrotny odczyt tabeli `Index Scan` zamiast trzech odczytów. Silnik raz skanuje dane dla zapytania głównego, a drugi raz wykonuje agregację za pomocą `Hash Match`, aby wyliczyć średnie dla kategorii, które następnie łączy operatorem `Hash Join`. Jest to o wiele wydajniejsze, ponieważ średnia dla danej kategorii liczona jest tylko raz i dołączana do pasujących produktów.

Funkcja okna



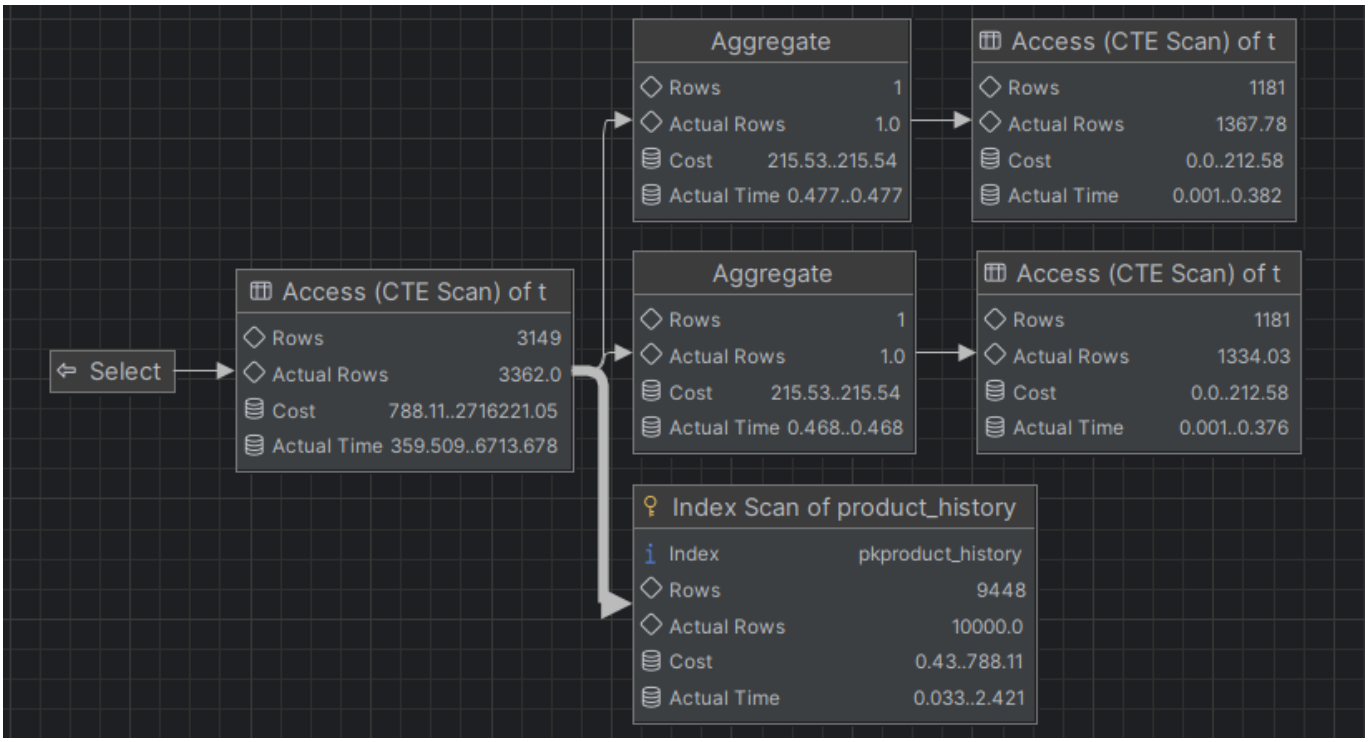
Koszt: 0.840

Czas: 30ms

W podejściu z funkcją okna silnik wykonuje tylko jeden odczyt tabeli `Index Scan`. Plan obejmuje sortowanie danych według kategorii, ich segmentację oraz buforowanie `Lazy Spool` w celu wyliczenia średniej przez operator `Stream Aggregate` i nałożenie końcowego filtra. Przy koszcie 0.840 czas wykonania wyniósł 30 ms. Jest to wynik znacznie lepszy niż w przypadku `podzapytania` (78 ms), lecz gorszy niż przy złączeniu `JOIN` (19 ms).

PostgreSql

Podzapytanie



Output | Plan

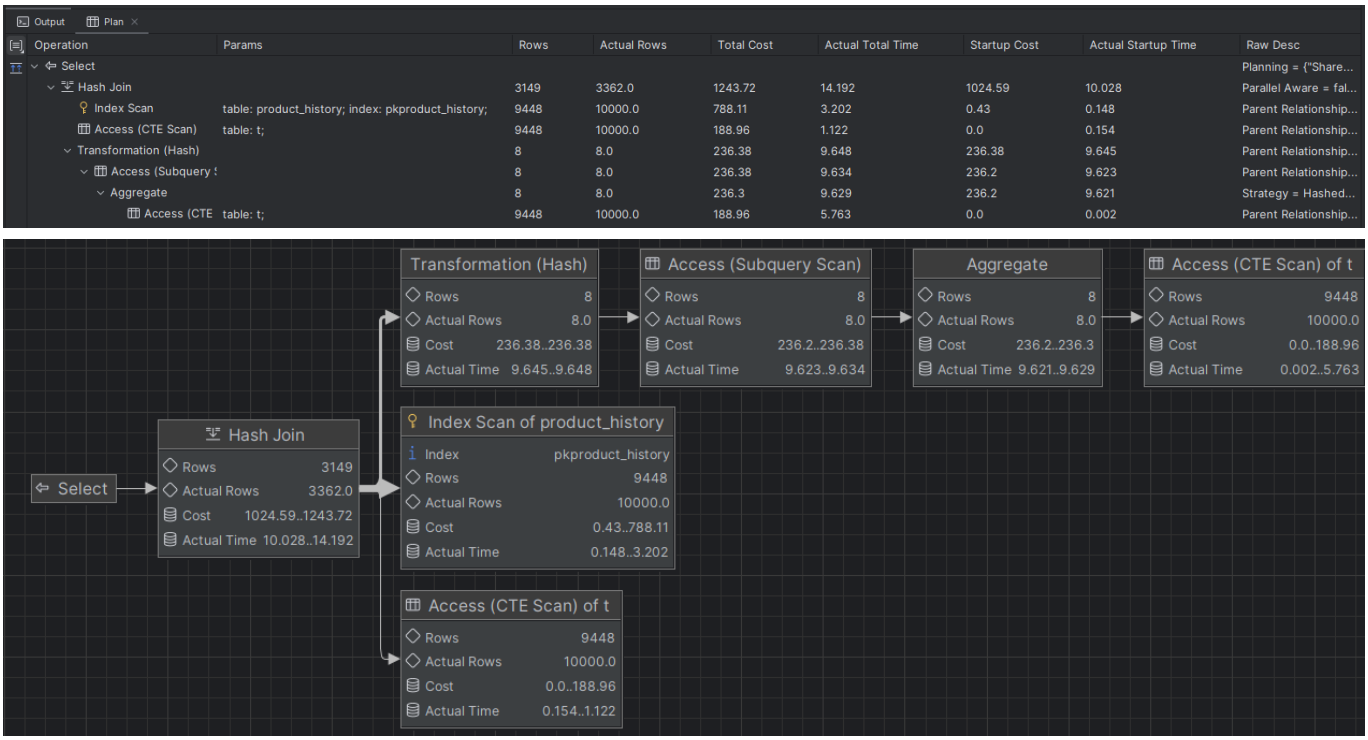
Operation	Params	Rows	Actual Rows	Total Cost	Actual Total Time	Startup Cost	Actual Startup Time	Raw Desc
Select								Planning = {"Shared...
Access (CTE Scan)	table: t;	3149	3362.0	2716221.05	6713.678	788.11	359.509	Parallel Aware = fal...
Index Scan	table: product_history; index: pkproduct_history;	9448	10000.0	788.11	2.421	0.43	0.033	Parent Relationship ...
Aggregate		1	1.0	215.54	0.468	215.53	0.468	Strategy = Plain;Par...
Access (CTE S	table: t;	1181	1334.03	212.58	0.376	0.0	0.001	Parent Relationship ...
Aggregate		1	1.0	215.54	0.477	215.53	0.477	Strategy = Plain;Par...
Access (CTE S	table: t;	1181	1367.78	212.58	0.382	0.0	0.001	Parent Relationship ...

Koszt: 2 716 221.05

Czas: 6 713.68ms

W przypadku PostgreSQL to rozwiązanie okazało się ekstremalnie niewydajne, co widać po gigantycznym koszcie przekraczającym 2,7 mln oraz czasie wykonania wynoszącym ponad 6,7 sekundy. Wynika to z faktu, że silnik wykonuje ogromną liczbę powtarzalnych operacji: dla każdego wiersza z zapytania głównego baza musi dwukrotnie przeszukać zbiór tymczasowy `CTE Scan` i wyliczyć średnią (osobno dla klauzuli SELECT i WHERE), co prowadzi do drastycznego spadku wydajności przy 10 000 rekordów.

Join

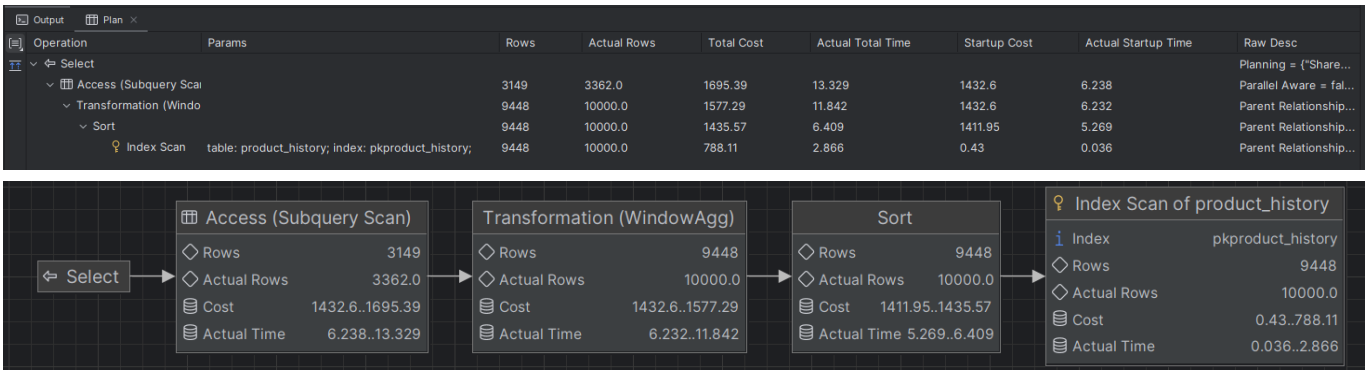


Koszt: 1243.72

Czas: 14.192ms

Zastosowanie złączenia `JOIN` w PostgreSQL przyniosło ogromną poprawę wydajności w porównaniu do podzapytania. Koszt spadł z milionów do 1243.72, a czas wykonania z ponad 6 sekund do zaledwie 14.192 ms. Silnik bazy danych wyliczył średnie dla kategorii na zbiorze CTE tylko raz za pomocą `Aggregate`, umieścił je w tymczasowej tablicy, a następnie błyskawicznie dopasował je do produktów za pomocą operatora `Hash Join`. Taka strategia eliminuje problem wielokrotnego skanowania tych samych danych dla każdego wiersza z osobna.

Funkcja okna



Koszt: 1695.39

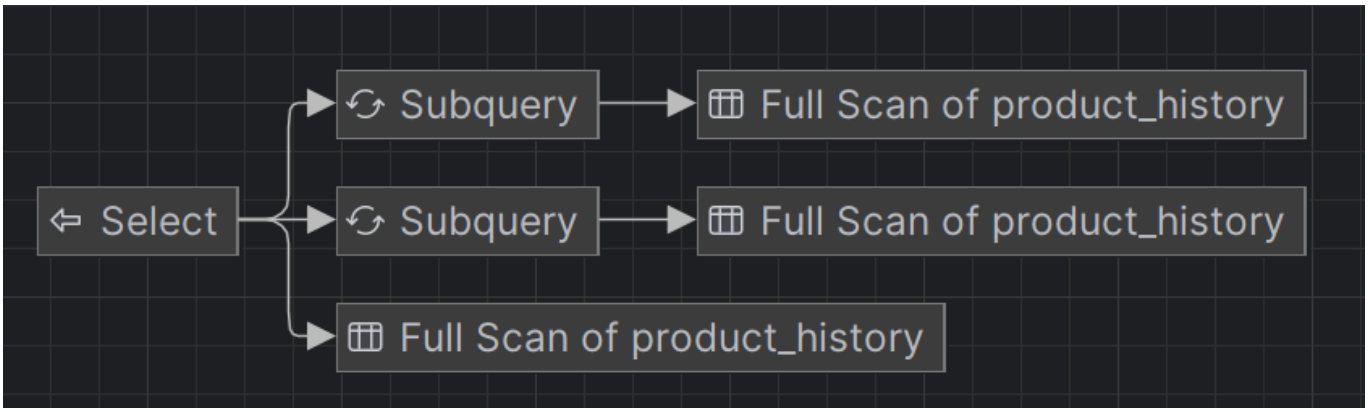
Czas: 13.329ms

W podejściu z `funkcją okna` PostgreSQL wykonuje tylko jeden odczyt tabeli `Index Scan`. Plan ma strukturę liniową: dane są sortowane według kategorii, a następnie operator `WindowAgg` wylicza średnią dla każdego okna danych. Ostatnim etapem jest filtrowanie wyników przez `Subquery Scan`. Przy koszcie 1695.39 czas

wykonania wyniósł 13.329 ms, co czyni tę metodę najszybszą w PostgreSQL, jest ona minimalnie wydajniejsza od złączenia `JOIN` (14.19 ms) i deklasuje podzapytanie, którego wykonanie trwało ponad 6 sekund.

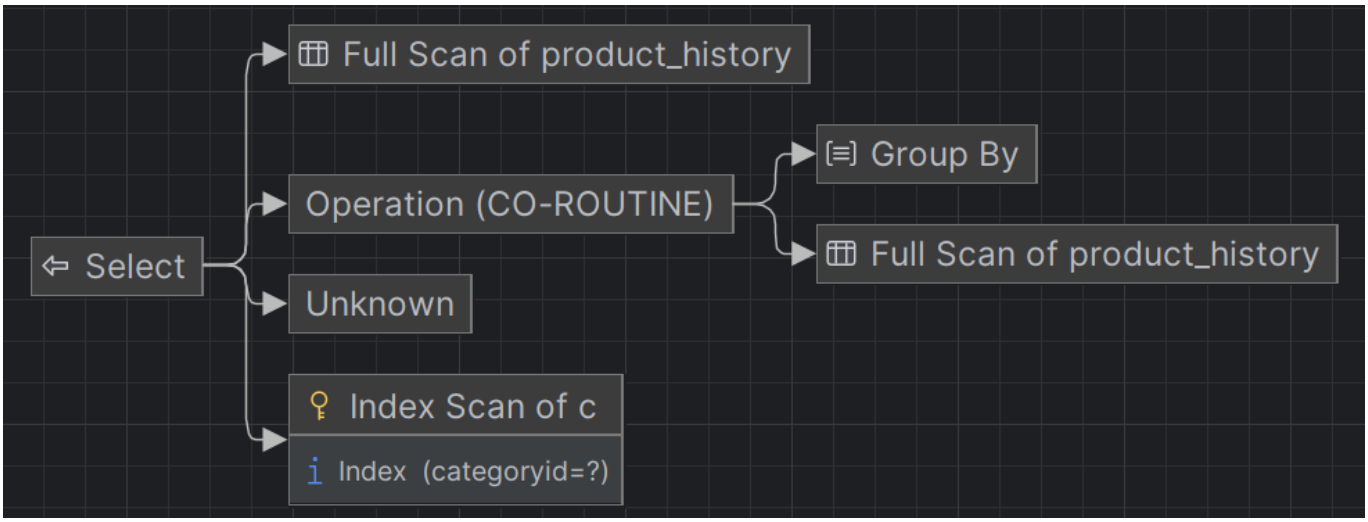
SQLite

Podzapytanie



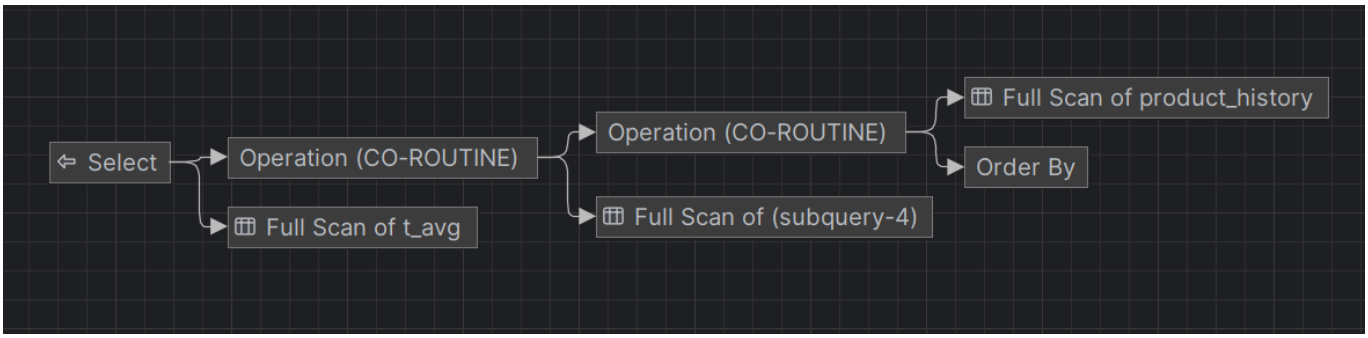
Plan wykonania w SQLite wykazuje bardzo niską wydajność ze względu na trzykrotne pełne skanowanie tabeli `Full Scan of product_history`. Silnik bazy danych oddzielnie skanuje tabelę dla zapytania głównego oraz dla każdego z dwóch podzapytań. Przy braku wykorzystania indeksów i dużej liczbie wierszy, takie podejście wymusza wielokrotne, nadmiarowe czytanie tych samych danych z dysku, co bardzo spowalnia zapytanie.

Join



Zastosowanie JOIN w SQLite jest znacznie wydajniejsze dzięki mechanizmowi `Operation (CO-ROUTINE)`, który wylicza średnie kategorii tylko raz. Silnik wykonuje jeden pełny odczyt tabeli w celu pogrupowania danych, tworząc tymczasowy zbiór wyników. Następnie łączy te gotowe wyliczenia z głównym odczytem tabeli za pomocą szybkiego przeszukiwania indeksu `Index Scan` na kluczu `categoryid`. Takie podejście eliminuje nadmiarowe skanowanie tabeli charakterystyczne dla podzapytań skorelowanych.

Funkcja okna



Z planu wynika, że silnik najpierw jednokrotnie skanuje i sortuje dane na potrzeby klauzuli `PARTITION BY`, po czym wylicza średnie w pamięci, tworząc wirtualną tabelę. Główne zapytanie wykonuje już tylko pojedynczy odczyt tych gotowych wyników `Full Scan of t_avg`, aby zastosować filtr z klauzuli `WHERE`. Takie podejście całkowicie eliminuje problem powtarzalnych obliczeń i wielokrotnego czytania dysku, zapewniając wysoką wydajność.

---

zadanie	pkt
1	1
2	1
3	1
4	1
5	1
6	2
razem:	7